

COMP132: Advanced Programming
Programming Project Report

Food Chain Through Time: Simulation
Design and Development

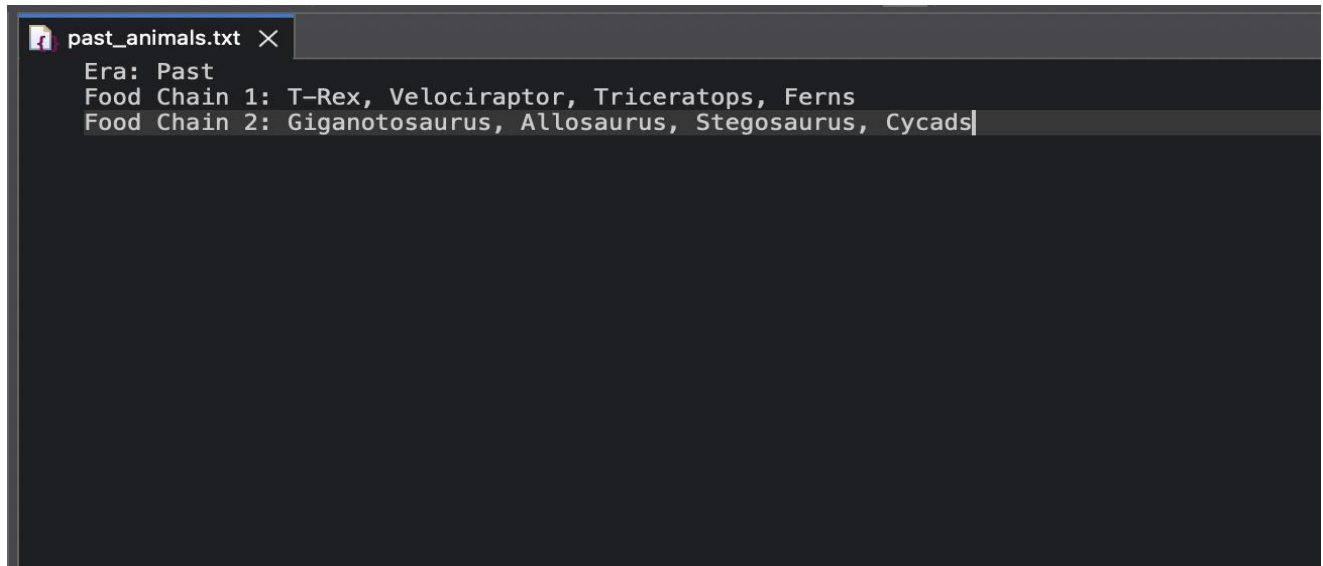
<Adnan Gültekin, 86816>

<Fall 2025>



PART 1

1.How the game changes with differences in animals.txt file?

A screenshot of a text editor window titled 'past_animals.txt'. The window has a dark background and a light-colored border. The text inside is as follows:

```
Era: Past
Food Chain 1: T-Rex, Velociraptor, Triceratops, Ferns
Food Chain 2: Giganotosaurus, Allosaurus, Stegosaurus, Cycads
```

"As you can see, there is a file named `past_animals.txt`. It is one of the three files in the instructions. We want the game to pick a random food chain from this file while playing. Then, it should assign these names to apex, predator, prey, and food. I did these assignments in the `getCharactersName(String selectedEra)` method inside the `FileIO` class."

```
public List<String> getCharacterNames(String selectedEra) {
    List<String> allFoodChains = new ArrayList<>();
    String fileName = selectedEra.toLowerCase() + "_animals.txt";

    try (Scanner scanner = new Scanner(new File(fileName))) {
        while (scanner.hasNextLine()) {
            String line = scanner.nextLine().trim();
            if (line.contains(":")) {
                allFoodChains.add(line);
            }
        }
    } catch (FileNotFoundException e) {
        System.err.println("CRITICAL ERROR: File not found -> " + fileName);
        return new ArrayList<>();
    }

    if (allFoodChains.isEmpty()) {
        return new ArrayList<>();
    }
    SecureRandom secureRandom = new SecureRandom();

    int randomIndex = secureRandom.nextInt(allFoodChains.size());
    String chosenLine = allFoodChains.get(randomIndex);

    String afterColon = chosenLine.split(":")[1].trim();
    String[] rawNamesArray = afterColon.split(",");

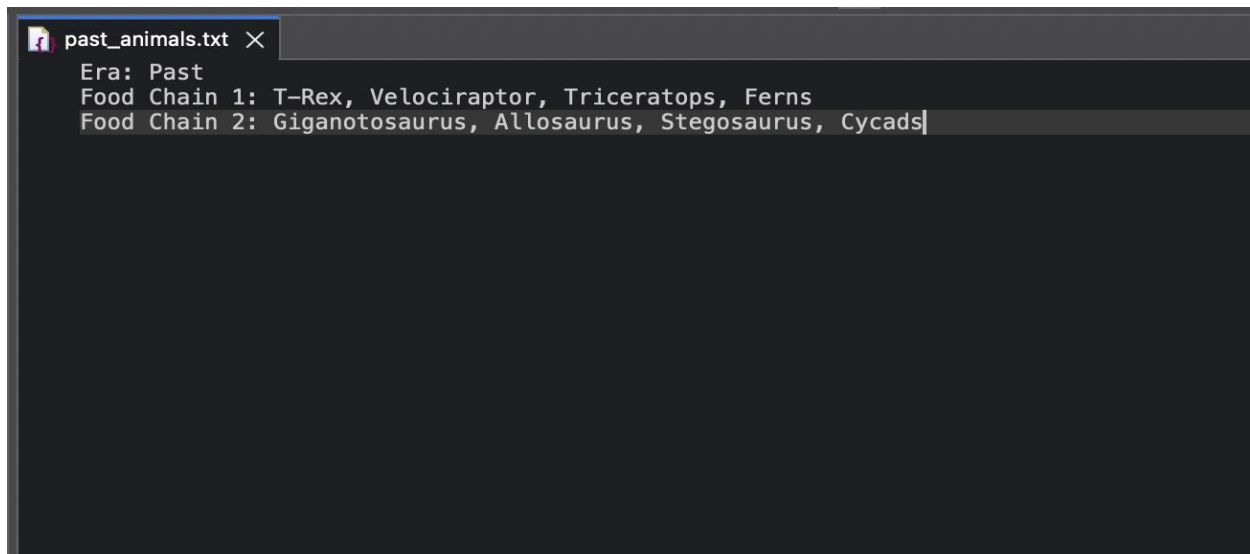
    List<String> cleanedNamesList = new ArrayList<>();

    for (String rawName : rawNamesArray) {
        cleanedNamesList.add(rawName.trim());
    }

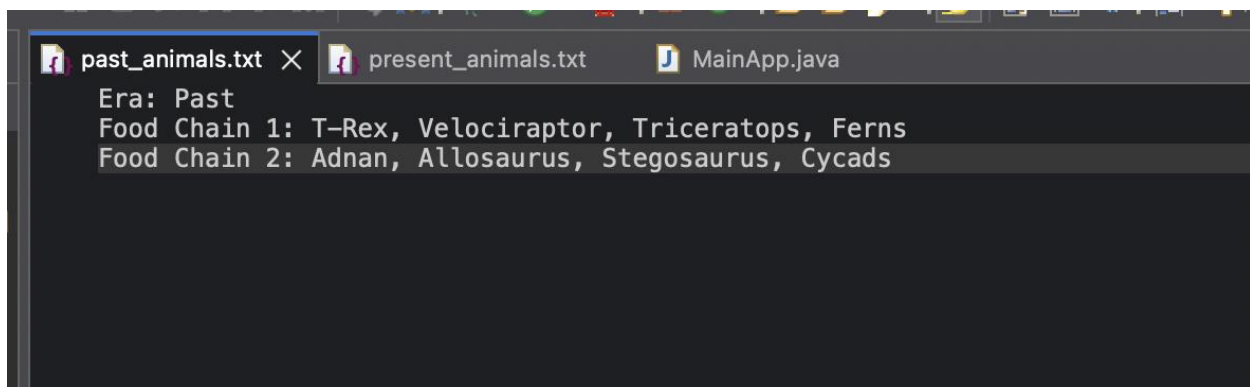
    return cleanedNamesList;
}
```

As seen in my screenshot, the method first takes the `era` as a parameter. Then, according to this era, it creates the file name generically by using `selectedEra.toLowerCase() + "_animals.txt"` instead of writing a specific hardcopy name. For example, if `selectedEra` is "past", it selects one of the food chains in the `past_animals.txt` file.

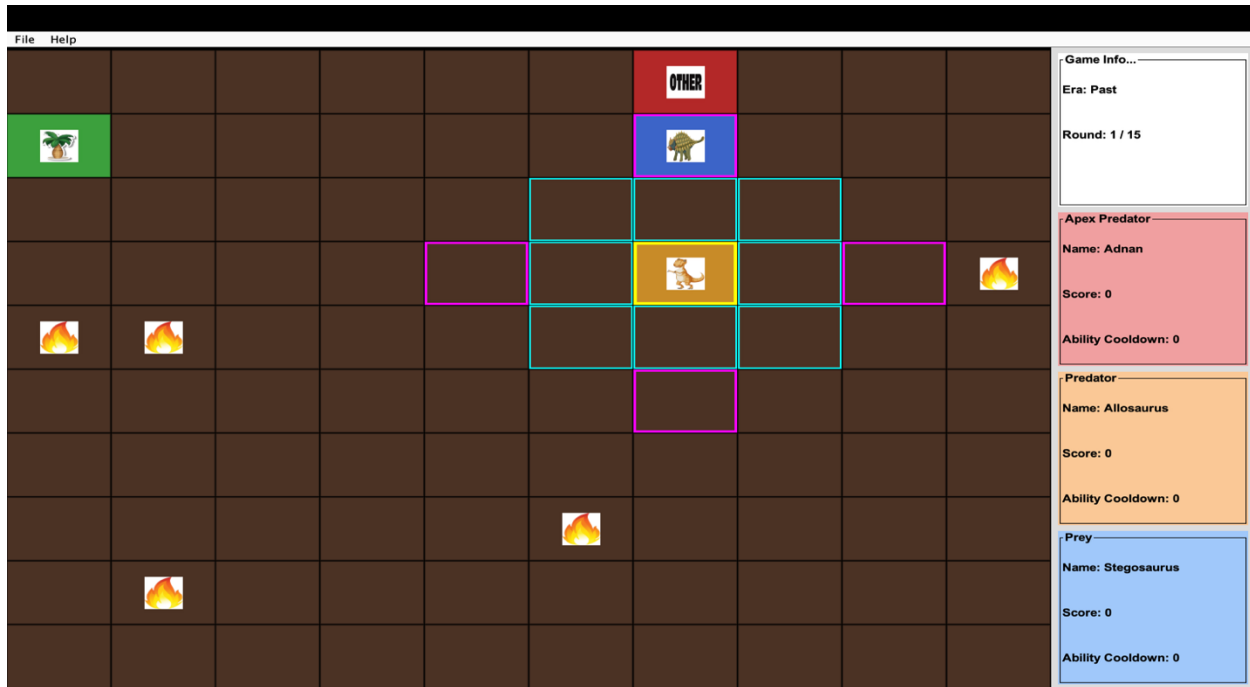
Then, inside the try-catch block, we create a `Scanner` object to read the file. In the next line, we use a while loop to read line by line, and if there is a ":" symbol, we add it to the `allFoodChains` `ArrayList`. In the catch part, I use `FileNotFoundException` to prevent errors if the file is not found. In the rest of the code, I create a `SecureRandom` object to choose one of the food chains randomly.



```
past_animals.txt X
Era: Past
Food Chain 1: T-Rex, Velociraptor, Triceratops, Ferns
Food Chain 2: Giganotosaurus, Allosaurus, Stegosaurus, Cycads|
```



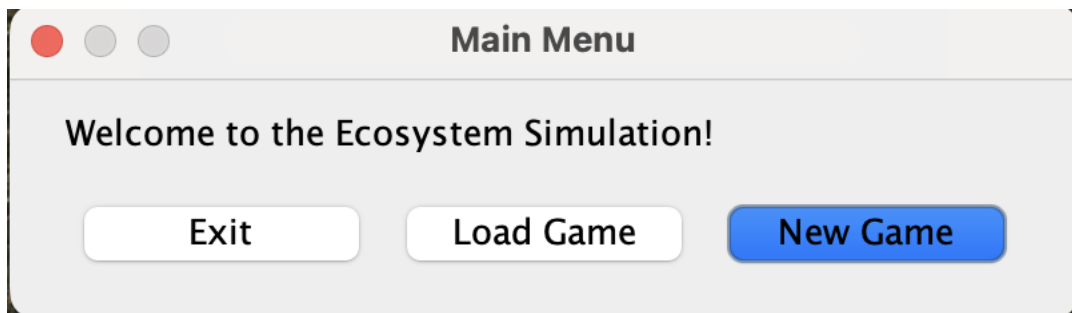
```
past_animals.txt X present_animals.txt MainApp.java
Era: Past
Food Chain 1: T-Rex, Velociraptor, Triceratops, Ferns
Food Chain 2: Adnan, Allosaurus, Stegosaurus, Cycads
```

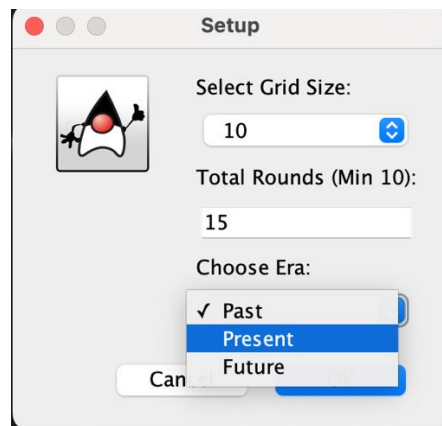
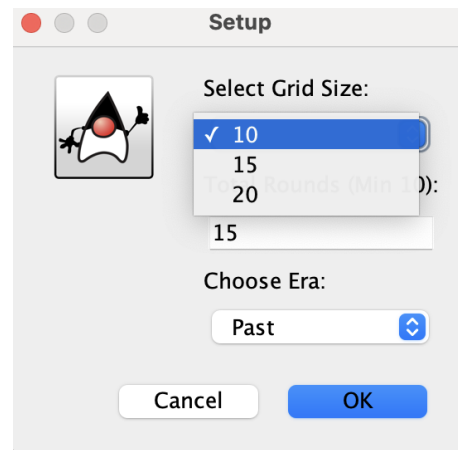
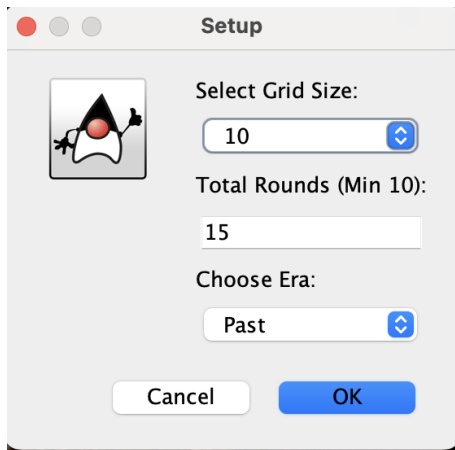


As you can see from the screenshots, even if I write "Adnan" instead of "Giganotosaurus", the game is still playable. It is clearly seen that "Adnan" is the ApexPredator because the name "Adnan" is written where "Giganotosaurus" used to be and the background is still red. Since application could not find a fileName as Adnan, it selects the other.png file automatically as a result of algorithm in GridCell class. Even if, all of the images change since the background color identifies the characters what their roles are it does not change the logic of game.

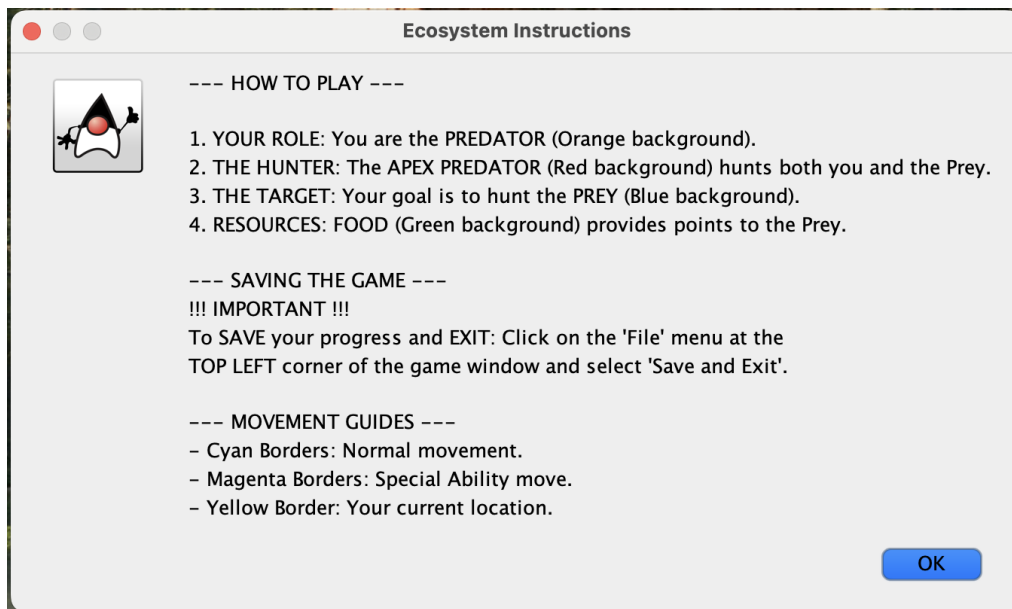
2-) How to start a new game from the GUI?

"First, we go to the main class and run the code. Then, a screen opens and we decide to start a new game. When we start a new game, we choose the Grid Size, the number of rounds, and which era we want to play."

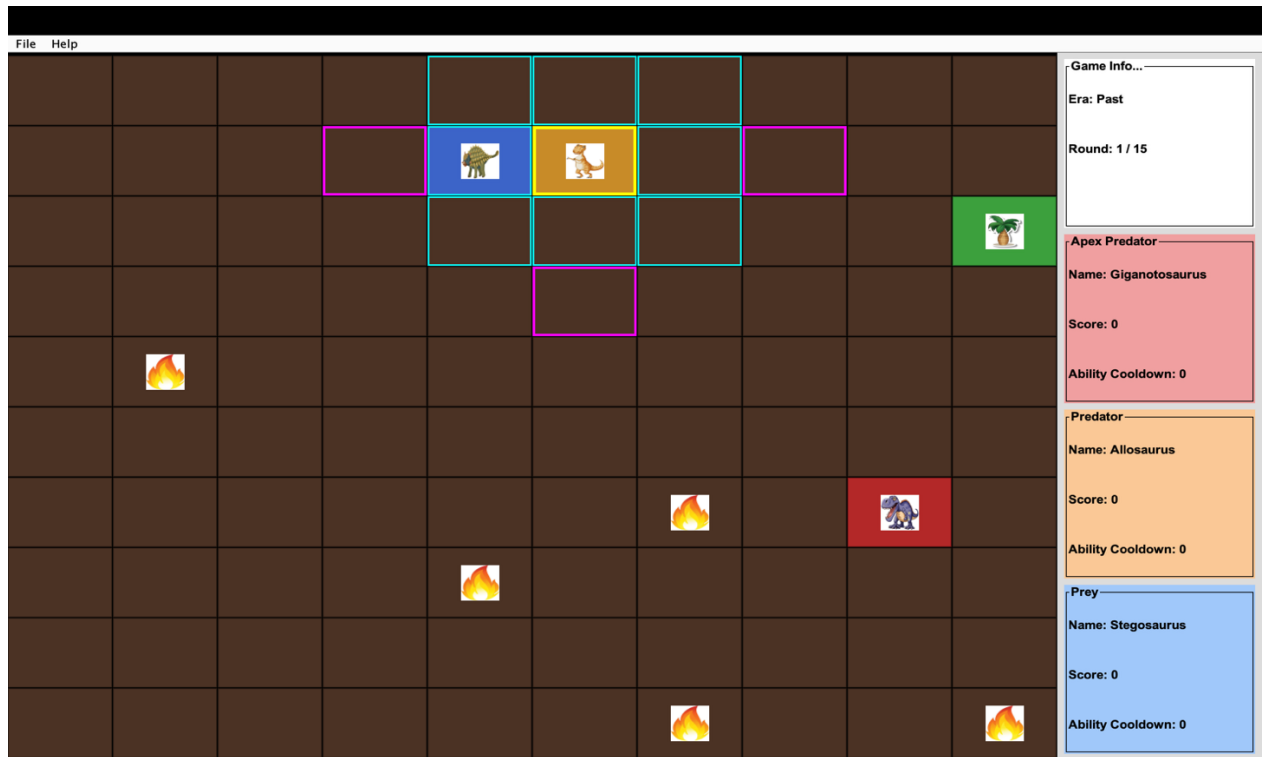




- After that, an info screen appears. This screen explains the main logic of the game, the colors, movements, and how to save the game.

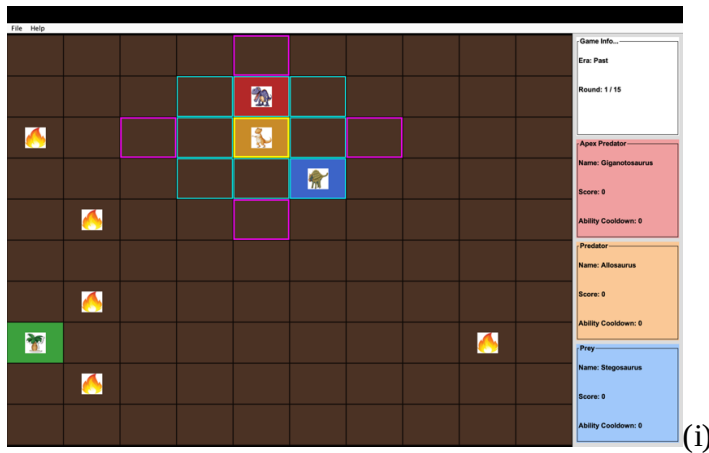


- After clicking the OK button, the game starts. The colored squares show where the player can move. Cyan squares are for walking, and purple squares are for using a special ability.

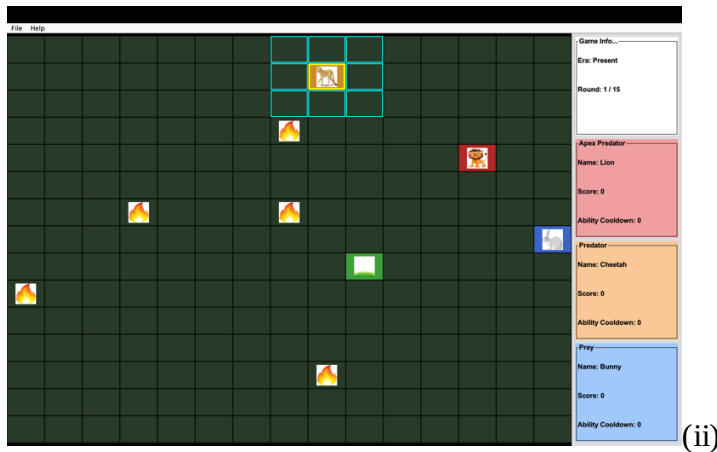


You can see the grid size and different versions of the eras in three different ways in the screenshots below.

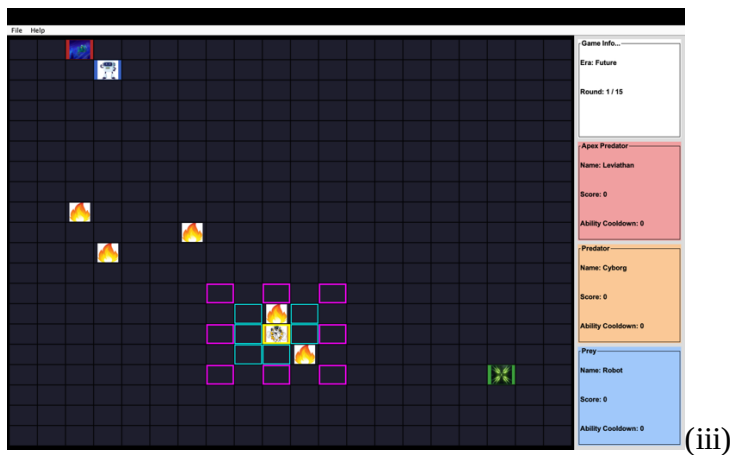
- 10x10 Grid Size, past era;
- 15x15 Grid Size, present era;
- 20x20 Grid Size, future era;



(i)

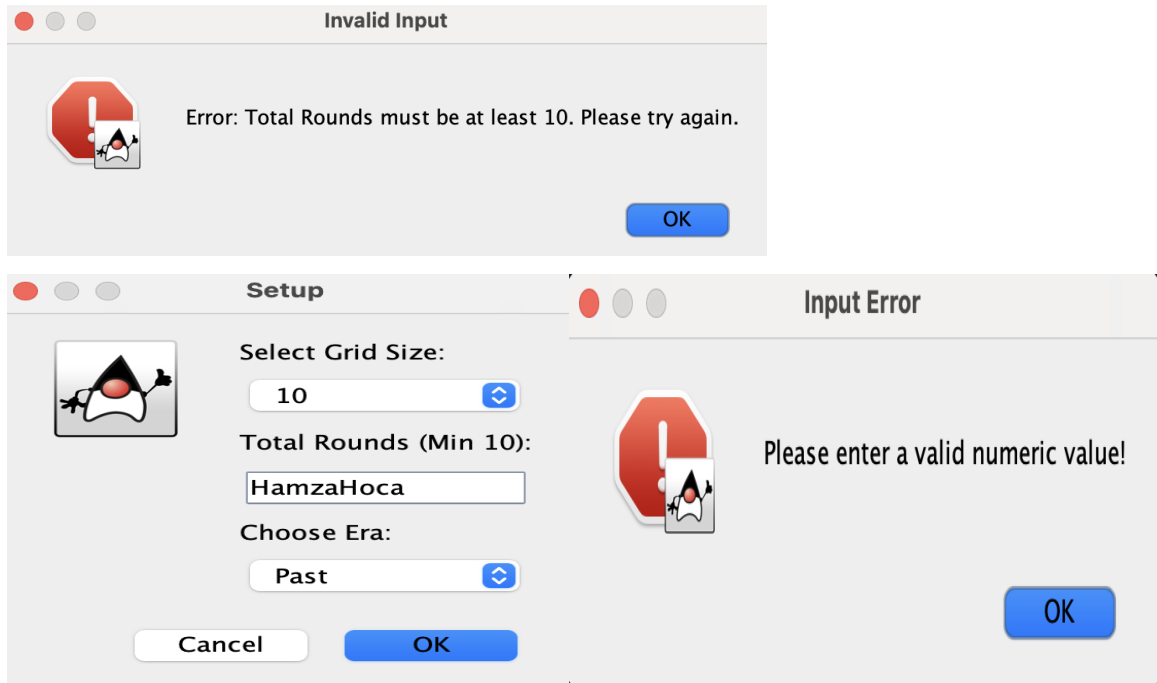


(ii)



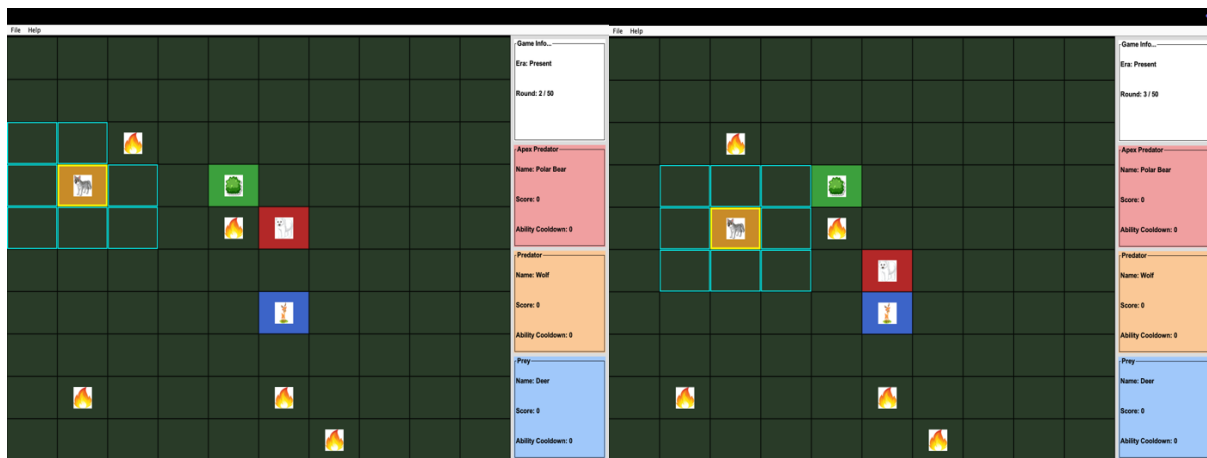
(iii)

- If the number of rounds entered is less than 10, as stated in the PDF, the system asks the user to enter it again. When 10 or more rounds are entered, the game starts without any problems. Also, if a string is entered instead of a number, a separate warning appears saying that the round number must be numeric.

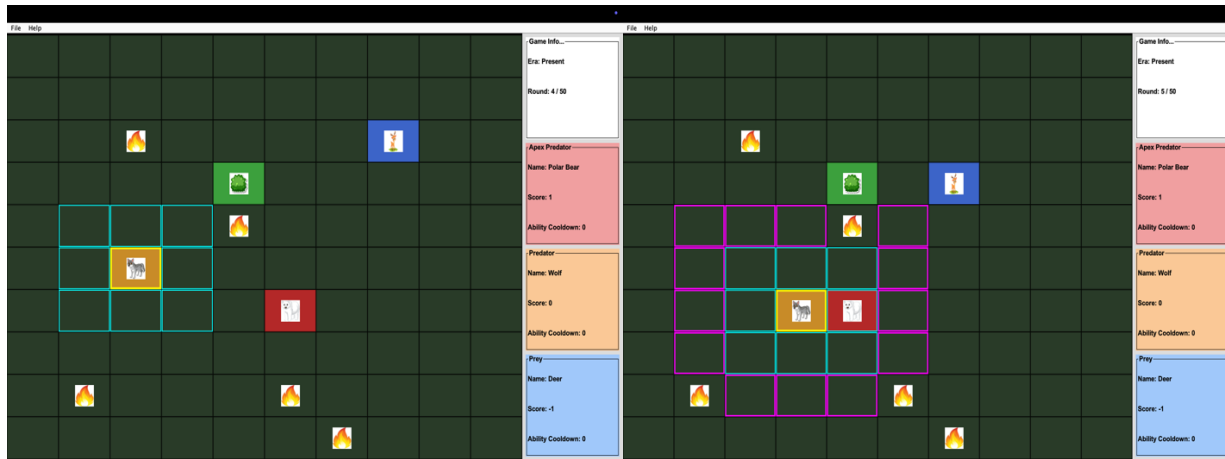


3-) Play a game, win the game, and show:

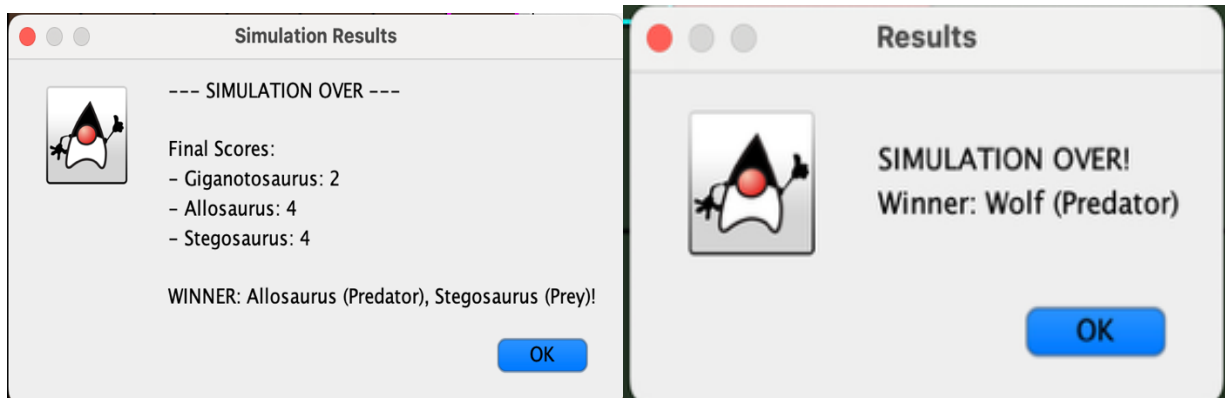
a-) The player (Predator) can move without any problem when the user clicks with the left mouse button on playable squares. Squares marked with a fire icon are obstacles, and characters cannot move onto these fire squares.



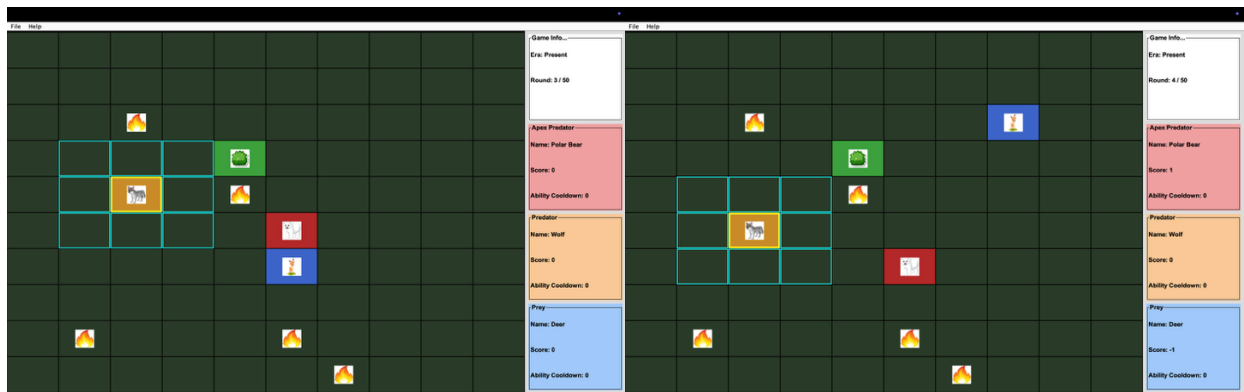
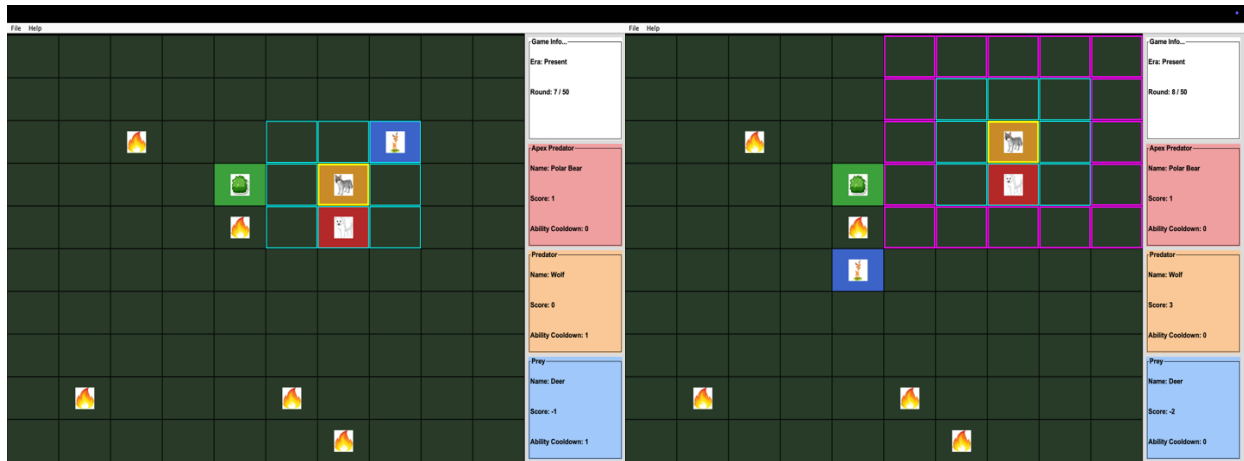
b-) Special moves: The squares where the Predator can use its special movement are shown in purple, while the other squares are shown in cyan. Also, the ability cooldown must be 0 to use the special ability.

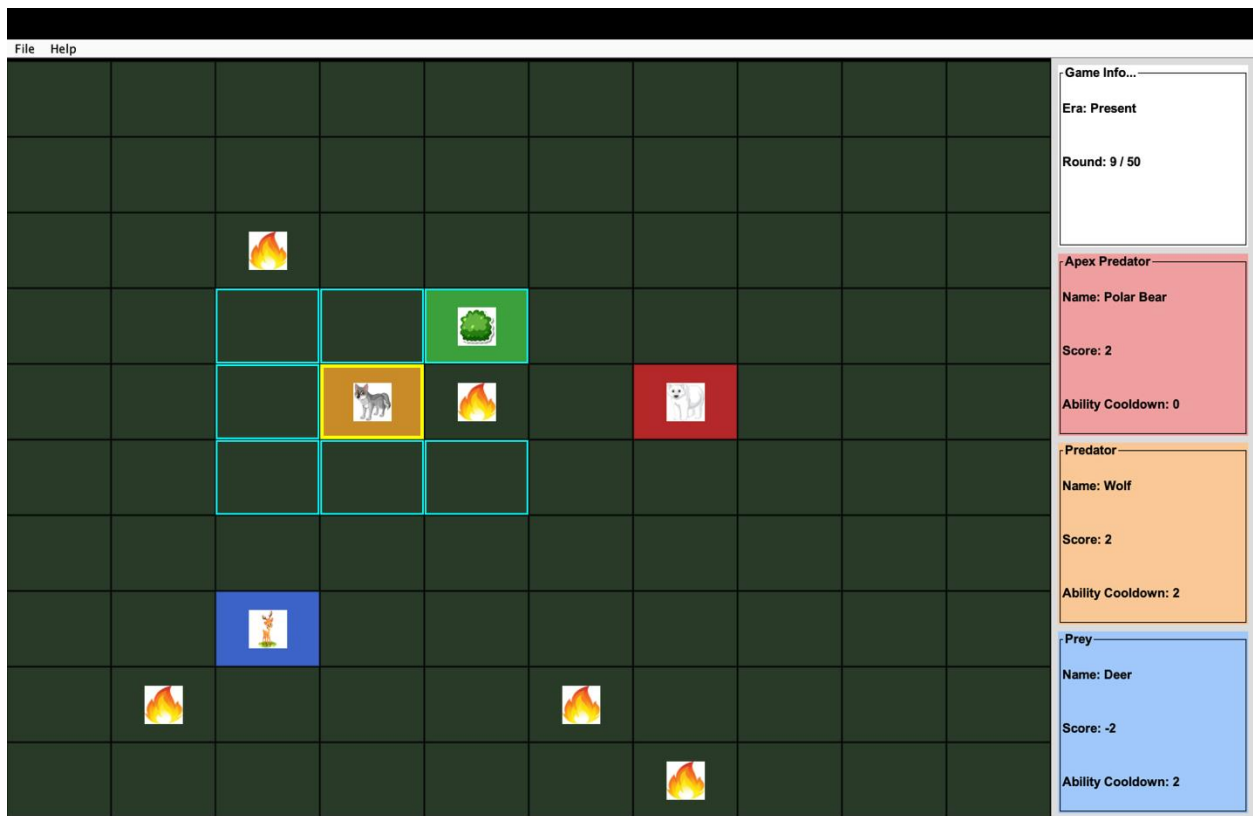
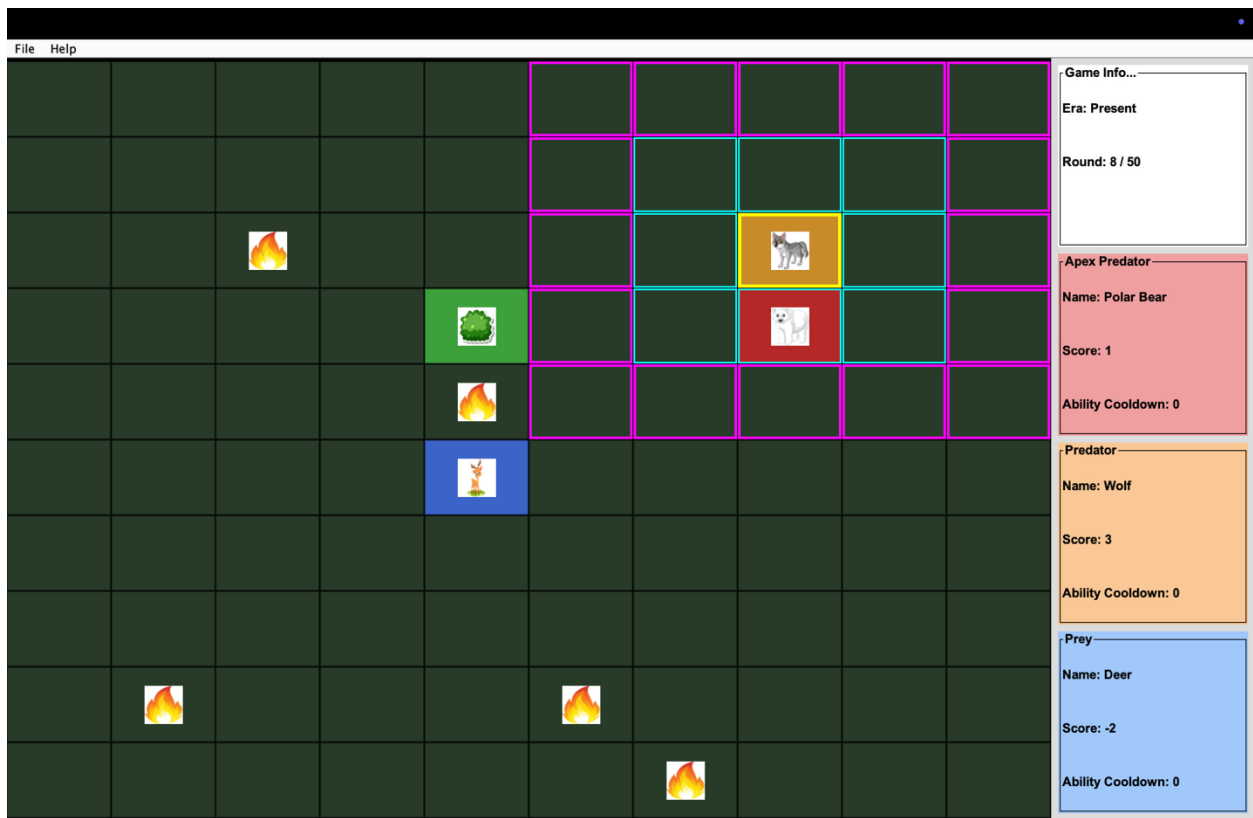


c-) Game completion: The game ends when the number of rounds entered on the start screen is finished. When the game ends, the character with the highest score wins the game. If more than one character has the same highest score, they all win.



d-) Point changes when prey is consumed: When prey is consumed prey's score decreases 1 and hunter's score increases 3 if it is predator and increases 1 if it is apex predator.





e-) Respawn behavior: Because Food, Prey, and Predator can be eaten by another hunter, they respawn randomly on an empty square of the grid after they die.

i-) Food

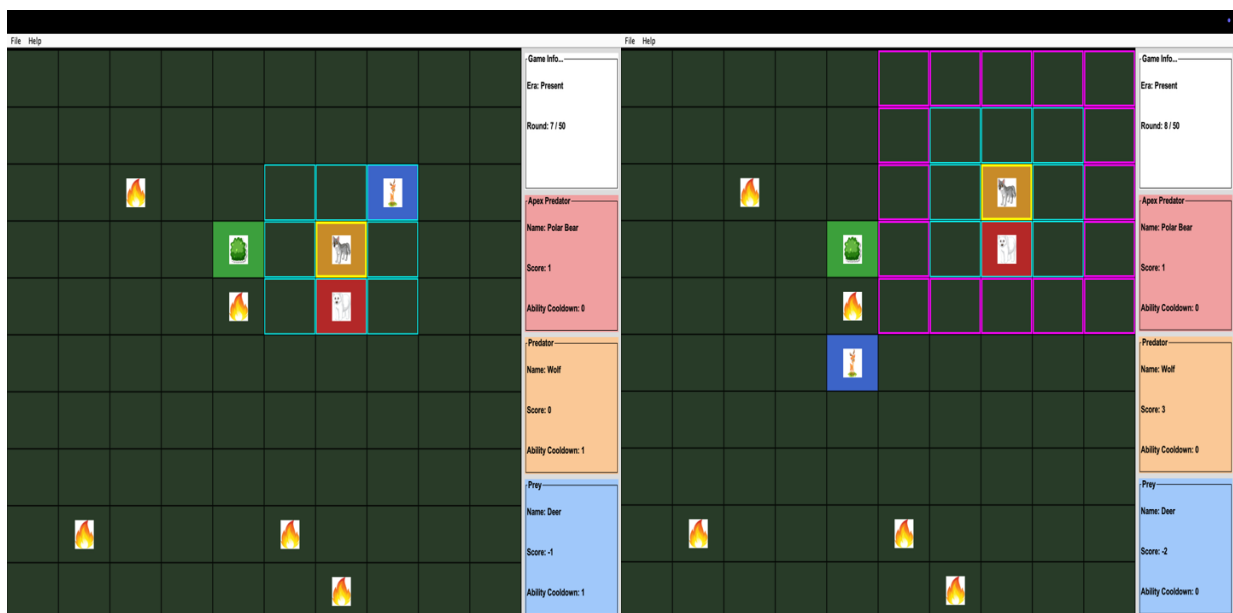
ii-) Prey

iii-) Predator

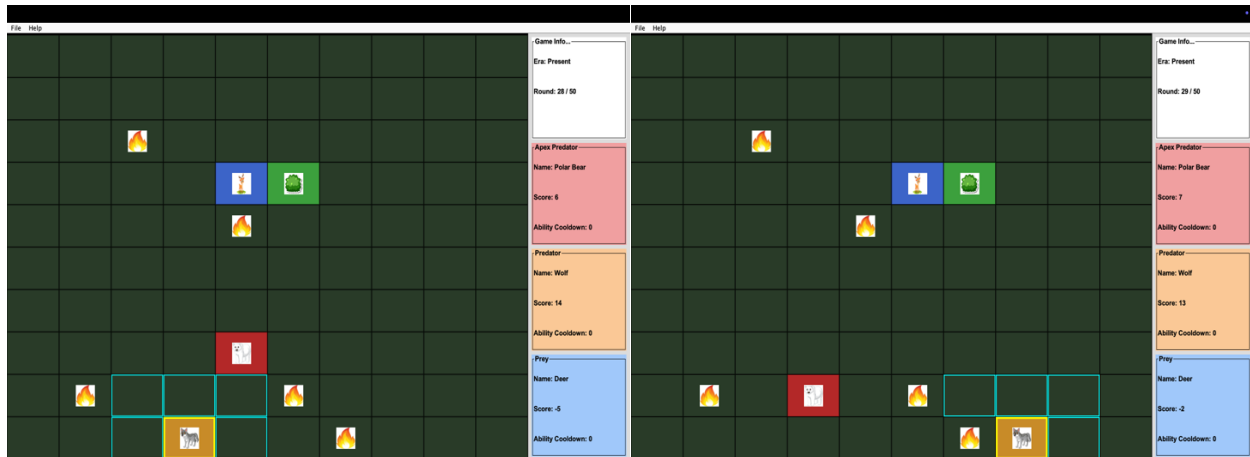
i-)



ii-)



iii-)



f-) Logs for the current game: Everything that happened in the game saves to log.txt file:

```
Sun Jan 04 22:56:34 TRT 2026] [ERA: Present] [ROUND: 1/50] INFO: === Round 1
Started ===
[Sun Jan 04 22:56:34 TRT 2026] [ERA: Present] [ROUND: 1/50] MOVE: Polar Bear
moved to: (4,5)
[Sun Jan 04 22:56:34 TRT 2026] [ERA: Present] [ROUND: 1/50] MOVE: Deer moved
to: (6,5)
[Sun Jan 04 22:56:34 TRT 2026] [ERA: Present] [ROUND: 1/50] INFO: === Round 1
Ended ===
[Sun Jan 04 22:56:41 TRT 2026] [ERA: Present] [ROUND: 2/50] INFO: === Round 2
Started ===
[Sun Jan 04 22:56:41 TRT 2026] [ERA: Present] [ROUND: 2/50] MOVE: Polar Bear
moved to: (5,5)
[Sun Jan 04 22:56:41 TRT 2026] [ERA: Present] [ROUND: 2/50] MOVE: Deer moved
to: (6,5)
[Sun Jan 04 22:56:41 TRT 2026] [ERA: Present] [ROUND: 2/50] INFO: === Round 2
Ended ===
[Sun Jan 04 22:56:47 TRT 2026] [ERA: Present] [ROUND: 3/50] INFO: === Round 3
Started ===
[Sun Jan 04 22:56:47 TRT 2026] [ERA: Present] [ROUND: 3/50] MOVE: Polar Bear
moved to: (6,5)
[Sun Jan 04 22:56:47 TRT 2026] [ERA: Present] [ROUND: 3/50] MOVE: Deer moved
to: (6,5)
[Sun Jan 04 22:56:47 TRT 2026] [ERA: Present] [ROUND: 3/50] COLLISION: Apex
Predator ate Prey.
[Sun Jan 04 22:56:47 TRT 2026] [ERA: Present] [ROUND: 3/50] INFO: === Round 3
Ended ===
[Sun Jan 04 22:57:29 TRT 2026] [ERA: Present] [ROUND: 4/50] INFO: === Round 4
Started ===
[Sun Jan 04 22:57:29 TRT 2026] [ERA: Present] [ROUND: 4/50] MOVE: Polar Bear
moved to: (6,4)
[Sun Jan 04 22:57:29 TRT 2026] [ERA: Present] [ROUND: 4/50] MOVE: Deer moved
to: (3,6)
[Sun Jan 04 22:57:29 TRT 2026] [ERA: Present] [ROUND: 4/50] INFO: === Round 4
Ended ===
```

[Sun Jan 04 22:57:40 TRT 2026] [ERA: Present] [ROUND: 5/50] INFO: === Round 5
 Started ===
 [Sun Jan 04 22:57:40 TRT 2026] [ERA: Present] [ROUND: 5/50] MOVE: Polar Bear
 moved to: (5,5)
 [Sun Jan 04 22:57:40 TRT 2026] [ERA: Present] [ROUND: 5/50] MOVE: Deer moved
 to: (1,8)
 [Sun Jan 04 22:57:40 TRT 2026] [ERA: Present] [ROUND: 5/50] INFO: === Round 5
 Ended ===
 [Sun Jan 04 22:57:42 TRT 2026] [ERA: Present] [ROUND: 6/50] INFO: === Round 6
 Started ===
 [Sun Jan 04 22:57:42 TRT 2026] [ERA: Present] [ROUND: 6/50] MOVE: Polar Bear
 moved to: (4,6)
 [Sun Jan 04 22:57:42 TRT 2026] [ERA: Present] [ROUND: 6/50] MOVE: Deer moved
 to: (2,7)
 [Sun Jan 04 22:57:42 TRT 2026] [ERA: Present] [ROUND: 6/50] INFO: === Round 6
 Ended ===
 [Sun Jan 04 22:57:46 TRT 2026] [ERA: Present] [ROUND: 7/50] COLLISION:
 Predator ate Prey.
 [Sun Jan 04 22:57:46 TRT 2026] [ERA: Present] [ROUND: 7/50] INFO: === Round 7
 Started ===
 [Sun Jan 04 22:57:46 TRT 2026] [ERA: Present] [ROUND: 7/50] MOVE: Polar Bear
 moved to: (3,7)
 [Sun Jan 04 22:57:46 TRT 2026] [ERA: Present] [ROUND: 7/50] MOVE: Deer moved
 to: (5,4)
 [Sun Jan 04 22:57:46 TRT 2026] [ERA: Present] [ROUND: 7/50] INFO: === Round 7
 Ended ===
 [Sun Jan 04 22:58:08 TRT 2026] [ERA: Present] [ROUND: 8/50] INFO: === Round 8
 Started ===
 [Sun Jan 04 22:58:08 TRT 2026] [ERA: Present] [ROUND: 8/50] MOVE: Polar Bear
 moved to: (4,6)
 [Sun Jan 04 22:58:08 TRT 2026] [ERA: Present] [ROUND: 8/50] MOVE: Deer moved
 to: (7,2)
 [Sun Jan 04 22:58:08 TRT 2026] [ERA: Present] [ROUND: 8/50] COLLISION: Apex
 Predator ate Predator.
 [Sun Jan 04 22:58:08 TRT 2026] [ERA: Present] [ROUND: 8/50] INFO: === Round 8
 Ended ===
 [Sun Jan 04 22:58:14 TRT 2026] [ERA: Present] [ROUND: 9/50] INFO: === Round 9
 Started ===
 [Sun Jan 04 22:58:14 TRT 2026] [ERA: Present] [ROUND: 9/50] MOVE: Polar Bear
 moved to: (5,5)
 [Sun Jan 04 22:58:14 TRT 2026] [ERA: Present] [ROUND: 9/50] MOVE: Deer moved
 to: (6,3)
 [Sun Jan 04 22:58:14 TRT 2026] [ERA: Present] [ROUND: 9/50] INFO: === Round 9
 Ended ===
 [Sun Jan 04 22:58:15 TRT 2026] [ERA: Present] [ROUND: 10/50] COLLISION:
 Predator ate Prey.
 [Sun Jan 04 22:58:15 TRT 2026] [ERA: Present] [ROUND: 10/50] INFO: === Round
 10 Started ===
 [Sun Jan 04 22:58:15 TRT 2026] [ERA: Present] [ROUND: 10/50] MOVE: Polar Bear
 moved to: (6,4)
 [Sun Jan 04 22:58:15 TRT 2026] [ERA: Present] [ROUND: 10/50] MOVE: Deer moved
 to: (5,3)
 [Sun Jan 04 22:58:15 TRT 2026] [ERA: Present] [ROUND: 10/50] INFO: === Round
 10 Ended ===
 [Sun Jan 04 22:58:16 TRT 2026] [ERA: Present] [ROUND: 11/50] COLLISION:
 Predator ate Prey.

[Sun Jan 04 22:58:16 TRT 2026] [ERA: Present] [ROUND: 11/50] INFO: === Round 11 Started ===
[Sun Jan 04 22:58:16 TRT 2026] [ERA: Present] [ROUND: 11/50] MOVE: Polar Bear moved to: (5,3)
[Sun Jan 04 22:58:16 TRT 2026] [ERA: Present] [ROUND: 11/50] MOVE: Deer moved to: (8,8)
[Sun Jan 04 22:58:16 TRT 2026] [ERA: Present] [ROUND: 11/50] COLLISION: Apex Predator ate Predator.
[Sun Jan 04 22:58:16 TRT 2026] [ERA: Present] [ROUND: 11/50] INFO: === Round 11 Ended ===
[Sun Jan 04 22:58:17 TRT 2026] [ERA: Present] [ROUND: 12/50] INFO: === Round 12 Started ===
[Sun Jan 04 22:58:17 TRT 2026] [ERA: Present] [ROUND: 12/50] MOVE: Polar Bear moved to: (5,3)
[Sun Jan 04 22:58:17 TRT 2026] [ERA: Present] [ROUND: 12/50] MOVE: Deer moved to: (7,7)
[Sun Jan 04 22:58:17 TRT 2026] [ERA: Present] [ROUND: 12/50] INFO: === Round 12 Ended ===
[Sun Jan 04 22:58:18 TRT 2026] [ERA: Present] [ROUND: 13/50] INFO: === Round 13 Started ===
[Sun Jan 04 22:58:18 TRT 2026] [ERA: Present] [ROUND: 13/50] MOVE: Polar Bear moved to: (6,4)
[Sun Jan 04 22:58:18 TRT 2026] [ERA: Present] [ROUND: 13/50] MOVE: Deer moved to: (9,9)
[Sun Jan 04 22:58:18 TRT 2026] [ERA: Present] [ROUND: 13/50] INFO: === Round 13 Ended ===
[Sun Jan 04 22:58:18 TRT 2026] [ERA: Present] [ROUND: 14/50] INFO: === Round 14 Started ===
[Sun Jan 04 22:58:18 TRT 2026] [ERA: Present] [ROUND: 14/50] MOVE: Polar Bear moved to: (5,5)
[Sun Jan 04 22:58:18 TRT 2026] [ERA: Present] [ROUND: 14/50] MOVE: Deer moved to: (8,8)
[Sun Jan 04 22:58:18 TRT 2026] [ERA: Present] [ROUND: 14/50] INFO: === Round 14 Ended ===
[Sun Jan 04 22:58:19 TRT 2026] [ERA: Present] [ROUND: 15/50] INFO: === Round 15 Started ===
[Sun Jan 04 22:58:19 TRT 2026] [ERA: Present] [ROUND: 15/50] MOVE: Polar Bear moved to: (6,6)
[Sun Jan 04 22:58:19 TRT 2026] [ERA: Present] [ROUND: 15/50] MOVE: Deer moved to: (7,7)
[Sun Jan 04 22:58:19 TRT 2026] [ERA: Present] [ROUND: 15/50] INFO: === Round 15 Ended ===
[Sun Jan 04 22:58:20 TRT 2026] [ERA: Present] [ROUND: 16/50] INFO: === Round 16 Started ===
[Sun Jan 04 22:58:20 TRT 2026] [ERA: Present] [ROUND: 16/50] MOVE: Polar Bear moved to: (7,7)
[Sun Jan 04 22:58:20 TRT 2026] [ERA: Present] [ROUND: 16/50] MOVE: Deer moved to: (7,7)
[Sun Jan 04 22:58:20 TRT 2026] [ERA: Present] [ROUND: 16/50] COLLISION: Apex Predator ate Prey.
[Sun Jan 04 22:58:20 TRT 2026] [ERA: Present] [ROUND: 16/50] INFO: === Round 16 Ended ===
[Sun Jan 04 22:58:21 TRT 2026] [ERA: Present] [ROUND: 17/50] INFO: === Round 17 Started ===
[Sun Jan 04 22:58:21 TRT 2026] [ERA: Present] [ROUND: 17/50] MOVE: Polar Bear moved to: (6,8)

[Sun Jan 04 22:58:21 TRT 2026] [ERA: Present] [ROUND: 17/50] MOVE: Deer moved to: (1,5)
[Sun Jan 04 22:58:21 TRT 2026] [ERA: Present] [ROUND: 17/50] INFO: === Round 17 Ended ===
[Sun Jan 04 22:58:21 TRT 2026] [ERA: Present] [ROUND: 18/50] INFO: === Round 18 Started ===
[Sun Jan 04 22:58:21 TRT 2026] [ERA: Present] [ROUND: 18/50] MOVE: Polar Bear moved to: (5,7)
[Sun Jan 04 22:58:21 TRT 2026] [ERA: Present] [ROUND: 18/50] MOVE: Deer moved to: (0,3)
[Sun Jan 04 22:58:21 TRT 2026] [ERA: Present] [ROUND: 18/50] INFO: === Round 18 Ended ===
[Sun Jan 04 22:58:22 TRT 2026] [ERA: Present] [ROUND: 19/50] INFO: === Round 19 Started ===
[Sun Jan 04 22:58:22 TRT 2026] [ERA: Present] [ROUND: 19/50] MOVE: Polar Bear moved to: (4,6)
[Sun Jan 04 22:58:22 TRT 2026] [ERA: Present] [ROUND: 19/50] MOVE: Deer moved to: (1,4)
[Sun Jan 04 22:58:22 TRT 2026] [ERA: Present] [ROUND: 19/50] INFO: === Round 19 Ended ===
[Sun Jan 04 22:58:22 TRT 2026] [ERA: Present] [ROUND: 20/50] INFO: === Round 20 Started ===
[Sun Jan 04 22:58:22 TRT 2026] [ERA: Present] [ROUND: 20/50] MOVE: Polar Bear moved to: (3,5)
[Sun Jan 04 22:58:22 TRT 2026] [ERA: Present] [ROUND: 20/50] MOVE: Deer moved to: (2,4)
[Sun Jan 04 22:58:22 TRT 2026] [ERA: Present] [ROUND: 20/50] INFO: === Round 20 Ended ===
[Sun Jan 04 22:58:23 TRT 2026] [ERA: Present] [ROUND: 21/50] COLLISION: Predator ate Prey.
[Sun Jan 04 22:58:23 TRT 2026] [ERA: Present] [ROUND: 21/50] INFO: === Round 21 Started ===
[Sun Jan 04 22:58:23 TRT 2026] [ERA: Present] [ROUND: 21/50] MOVE: Polar Bear moved to: (2,4)
[Sun Jan 04 22:58:23 TRT 2026] [ERA: Present] [ROUND: 21/50] MOVE: Deer moved to: (6,7)
[Sun Jan 04 22:58:23 TRT 2026] [ERA: Present] [ROUND: 21/50] COLLISION: Apex Predator ate Predator.
[Sun Jan 04 22:58:23 TRT 2026] [ERA: Present] [ROUND: 21/50] INFO: === Round 21 Ended ===
[Sun Jan 04 22:58:24 TRT 2026] [ERA: Present] [ROUND: 22/50] INFO: === Round 22 Started ===
[Sun Jan 04 22:58:24 TRT 2026] [ERA: Present] [ROUND: 22/50] MOVE: Polar Bear moved to: (3,5)
[Sun Jan 04 22:58:24 TRT 2026] [ERA: Present] [ROUND: 22/50] MOVE: Deer moved to: (6,7)
[Sun Jan 04 22:58:24 TRT 2026] [ERA: Present] [ROUND: 22/50] INFO: === Round 22 Ended ===
[Sun Jan 04 22:58:25 TRT 2026] [ERA: Present] [ROUND: 23/50] INFO: === Round 23 Started ===
[Sun Jan 04 22:58:25 TRT 2026] [ERA: Present] [ROUND: 23/50] MOVE: Polar Bear moved to: (4,6)
[Sun Jan 04 22:58:25 TRT 2026] [ERA: Present] [ROUND: 23/50] MOVE: Deer moved to: (6,7)
[Sun Jan 04 22:58:25 TRT 2026] [ERA: Present] [ROUND: 23/50] INFO: === Round 23 Ended ===

[Sun Jan 04 22:58:26 TRT 2026] [ERA: Present] [ROUND: 24/50] COLLISION: Predator ate Prey.

[Sun Jan 04 22:58:26 TRT 2026] [ERA: Present] [ROUND: 24/50] INFO: === Round 24 Started ===

[Sun Jan 04 22:58:26 TRT 2026] [ERA: Present] [ROUND: 24/50] MOVE: Polar Bear moved to: (5,7)

[Sun Jan 04 22:58:26 TRT 2026] [ERA: Present] [ROUND: 24/50] MOVE: Deer moved to: (9,2)

[Sun Jan 04 22:58:26 TRT 2026] [ERA: Present] [ROUND: 24/50] INFO: === Round 24 Ended ===

[Sun Jan 04 22:58:27 TRT 2026] [ERA: Present] [ROUND: 25/50] INFO: === Round 25 Started ===

[Sun Jan 04 22:58:27 TRT 2026] [ERA: Present] [ROUND: 25/50] MOVE: Polar Bear moved to: (6,6)

[Sun Jan 04 22:58:27 TRT 2026] [ERA: Present] [ROUND: 25/50] MOVE: Deer moved to: (8,3)

[Sun Jan 04 22:58:27 TRT 2026] [ERA: Present] [ROUND: 25/50] INFO: === Round 25 Ended ===

[Sun Jan 04 22:58:28 TRT 2026] [ERA: Present] [ROUND: 26/50] INFO: === Round 26 Started ===

[Sun Jan 04 22:58:28 TRT 2026] [ERA: Present] [ROUND: 26/50] MOVE: Polar Bear moved to: (7,5)

[Sun Jan 04 22:58:28 TRT 2026] [ERA: Present] [ROUND: 26/50] MOVE: Deer moved to: (7,4)

[Sun Jan 04 22:58:28 TRT 2026] [ERA: Present] [ROUND: 26/50] INFO: === Round 26 Ended ===

[Sun Jan 04 22:58:28 TRT 2026] [ERA: Present] [ROUND: 27/50] COLLISION: Predator ate Prey.

[Sun Jan 04 22:58:28 TRT 2026] [ERA: Present] [ROUND: 27/50] INFO: === Round 27 Started ===

[Sun Jan 04 22:58:28 TRT 2026] [ERA: Present] [ROUND: 27/50] MOVE: Polar Bear moved to: (7,4)

[Sun Jan 04 22:58:28 TRT 2026] [ERA: Present] [ROUND: 27/50] MOVE: Deer moved to: (3,4)

[Sun Jan 04 22:58:28 TRT 2026] [ERA: Present] [ROUND: 27/50] COLLISION: Apex Predator ate Predator.

[Sun Jan 04 22:58:28 TRT 2026] [ERA: Present] [ROUND: 27/50] COLLISION: Prey ate Food.

[Sun Jan 04 22:58:28 TRT 2026] [ERA: Present] [ROUND: 27/50] INFO: === Round 27 Ended ===

[Sun Jan 04 22:58:33 TRT 2026] [ERA: Present] [ROUND: 28/50] INFO: === Round 28 Started ===

[Sun Jan 04 22:58:33 TRT 2026] [ERA: Present] [ROUND: 28/50] MOVE: Polar Bear moved to: (8,3)

[Sun Jan 04 22:58:33 TRT 2026] [ERA: Present] [ROUND: 28/50] MOVE: Deer moved to: (3,5)

[Sun Jan 04 22:58:33 TRT 2026] [ERA: Present] [ROUND: 28/50] COLLISION: Apex Predator ate Predator.

[Sun Jan 04 22:58:33 TRT 2026] [ERA: Present] [ROUND: 28/50] COLLISION: Prey ate Food.

[Sun Jan 04 22:58:33 TRT 2026] [ERA: Present] [ROUND: 28/50] INFO: === Round 28 Ended ===

[Sun Jan 04 22:58:38 TRT 2026] [ERA: Present] [ROUND: 29/50] INFO: === Round 29 Started ===

[Sun Jan 04 22:58:38 TRT 2026] [ERA: Present] [ROUND: 29/50] MOVE: Polar Bear moved to: (8,4)

[Sun Jan 04 22:58:38 TRT 2026] [ERA: Present] [ROUND: 29/50] MOVE: Deer moved to: (3,6)
 [Sun Jan 04 22:58:38 TRT 2026] [ERA: Present] [ROUND: 29/50] COLLISION: Prey ate Food.
 [Sun Jan 04 22:58:38 TRT 2026] [ERA: Present] [ROUND: 29/50] INFO: === Round 29 Ended ===
 [Sun Jan 04 22:58:39 TRT 2026] [ERA: Present] [ROUND: 30/50] INFO: === Round 30 Started ===
 [Sun Jan 04 22:58:39 TRT 2026] [ERA: Present] [ROUND: 30/50] MOVE: Polar Bear moved to: (7,5)
 [Sun Jan 04 22:58:39 TRT 2026] [ERA: Present] [ROUND: 30/50] MOVE: Deer moved to: (4,7)
 [Sun Jan 04 22:58:39 TRT 2026] [ERA: Present] [ROUND: 30/50] INFO: === Round 30 Ended ===
 [Sun Jan 04 22:58:39 TRT 2026] [ERA: Present] [ROUND: 31/50] INFO: === Round 31 Started ===
 [Sun Jan 04 22:58:39 TRT 2026] [ERA: Present] [ROUND: 31/50] MOVE: Polar Bear moved to: (6,6)
 [Sun Jan 04 22:58:39 TRT 2026] [ERA: Present] [ROUND: 31/50] MOVE: Deer moved to: (2,5)
 [Sun Jan 04 22:58:39 TRT 2026] [ERA: Present] [ROUND: 31/50] INFO: === Round 31 Ended ===
 [Sun Jan 04 22:58:40 TRT 2026] [ERA: Present] [ROUND: 32/50] INFO: === Round 32 Started ===
 [Sun Jan 04 22:58:40 TRT 2026] [ERA: Present] [ROUND: 32/50] MOVE: Polar Bear moved to: (5,7)
 [Sun Jan 04 22:58:40 TRT 2026] [ERA: Present] [ROUND: 32/50] MOVE: Deer moved to: (3,6)
 [Sun Jan 04 22:58:40 TRT 2026] [ERA: Present] [ROUND: 32/50] INFO: === Round 32 Ended ===
 [Sun Jan 04 22:58:42 TRT 2026] [ERA: Present] [ROUND: 33/50] INFO: === Round 33 Started ===
 [Sun Jan 04 22:58:42 TRT 2026] [ERA: Present] [ROUND: 33/50] MOVE: Polar Bear moved to: (4,6)
 [Sun Jan 04 22:58:42 TRT 2026] [ERA: Present] [ROUND: 33/50] MOVE: Deer moved to: (4,7)
 [Sun Jan 04 22:58:42 TRT 2026] [ERA: Present] [ROUND: 33/50] INFO: === Round 33 Ended ===
 [Sun Jan 04 22:58:43 TRT 2026] [ERA: Present] [ROUND: 34/50] INFO: === Round 34 Started ===
 [Sun Jan 04 22:58:43 TRT 2026] [ERA: Present] [ROUND: 34/50] MOVE: Polar Bear moved to: (4,7)
 [Sun Jan 04 22:58:43 TRT 2026] [ERA: Present] [ROUND: 34/50] MOVE: Deer moved to: (4,7)
 [Sun Jan 04 22:58:43 TRT 2026] [ERA: Present] [ROUND: 34/50] COLLISION: Apex Predator ate Prey.
 [Sun Jan 04 22:58:43 TRT 2026] [ERA: Present] [ROUND: 34/50] INFO: === Round 34 Ended ===
 [Sun Jan 04 22:58:44 TRT 2026] [ERA: Present] [ROUND: 35/50] INFO: === Round 35 Started ===
 [Sun Jan 04 22:58:44 TRT 2026] [ERA: Present] [ROUND: 35/50] MOVE: Polar Bear moved to: (5,8)
 [Sun Jan 04 22:58:44 TRT 2026] [ERA: Present] [ROUND: 35/50] MOVE: Deer moved to: (5,1)
 [Sun Jan 04 22:58:44 TRT 2026] [ERA: Present] [ROUND: 35/50] INFO: === Round 35 Ended ===

[Sun Jan 04 22:58:45 TRT 2026] [ERA: Present] [ROUND: 36/50] INFO: === Round
 36 Started ===
 [Sun Jan 04 22:58:45 TRT 2026] [ERA: Present] [ROUND: 36/50] MOVE: Polar Bear
 moved to: (6,9)
 [Sun Jan 04 22:58:45 TRT 2026] [ERA: Present] [ROUND: 36/50] MOVE: Deer moved
 to: (5,2)
 [Sun Jan 04 22:58:45 TRT 2026] [ERA: Present] [ROUND: 36/50] INFO: === Round
 36 Ended ===
 [Sun Jan 04 22:58:45 TRT 2026] [ERA: Present] [ROUND: 37/50] INFO: === Round
 37 Started ===
 [Sun Jan 04 22:58:45 TRT 2026] [ERA: Present] [ROUND: 37/50] MOVE: Polar Bear
 moved to: (7,8)
 [Sun Jan 04 22:58:45 TRT 2026] [ERA: Present] [ROUND: 37/50] MOVE: Deer moved
 to: (5,3)
 [Sun Jan 04 22:58:45 TRT 2026] [ERA: Present] [ROUND: 37/50] INFO: === Round
 37 Ended ===
 [Sun Jan 04 22:58:46 TRT 2026] [ERA: Present] [ROUND: 38/50] INFO: === Round
 38 Started ===
 [Sun Jan 04 22:58:46 TRT 2026] [ERA: Present] [ROUND: 38/50] MOVE: Polar Bear
 moved to: (6,7)
 [Sun Jan 04 22:58:46 TRT 2026] [ERA: Present] [ROUND: 38/50] MOVE: Deer moved
 to: (3,1)
 [Sun Jan 04 22:58:46 TRT 2026] [ERA: Present] [ROUND: 38/50] INFO: === Round
 38 Ended ===
 [Sun Jan 04 22:58:47 TRT 2026] [ERA: Present] [ROUND: 39/50] INFO: === Round
 39 Started ===
 [Sun Jan 04 22:58:47 TRT 2026] [ERA: Present] [ROUND: 39/50] MOVE: Polar Bear
 moved to: (5,6)
 [Sun Jan 04 22:58:47 TRT 2026] [ERA: Present] [ROUND: 39/50] MOVE: Deer moved
 to: (4,2)
 [Sun Jan 04 22:58:47 TRT 2026] [ERA: Present] [ROUND: 39/50] INFO: === Round
 39 Ended ===
 [Sun Jan 04 22:58:47 TRT 2026] [ERA: Present] [ROUND: 40/50] INFO: === Round
 40 Started ===
 [Sun Jan 04 22:58:47 TRT 2026] [ERA: Present] [ROUND: 40/50] MOVE: Polar Bear
 moved to: (5,5)
 [Sun Jan 04 22:58:47 TRT 2026] [ERA: Present] [ROUND: 40/50] MOVE: Deer moved
 to: (5,3)
 [Sun Jan 04 22:58:47 TRT 2026] [ERA: Present] [ROUND: 40/50] INFO: === Round
 40 Ended ===
 [Sun Jan 04 22:58:48 TRT 2026] [ERA: Present] [ROUND: 41/50] COLLISION:
 Predator ate Prey.
 [Sun Jan 04 22:58:48 TRT 2026] [ERA: Present] [ROUND: 41/50] INFO: === Round
 41 Started ===
 [Sun Jan 04 22:58:48 TRT 2026] [ERA: Present] [ROUND: 41/50] MOVE: Polar Bear
 moved to: (5,4)
 [Sun Jan 04 22:58:48 TRT 2026] [ERA: Present] [ROUND: 41/50] MOVE: Deer moved
 to: (2,5)
 [Sun Jan 04 22:58:48 TRT 2026] [ERA: Present] [ROUND: 41/50] INFO: === Round
 41 Ended ===
 [Sun Jan 04 22:58:49 TRT 2026] [ERA: Present] [ROUND: 42/50] INFO: === Round
 42 Started ===
 [Sun Jan 04 22:58:49 TRT 2026] [ERA: Present] [ROUND: 42/50] MOVE: Polar Bear
 moved to: (4,3)
 [Sun Jan 04 22:58:49 TRT 2026] [ERA: Present] [ROUND: 42/50] MOVE: Deer moved
 to: (0,7)

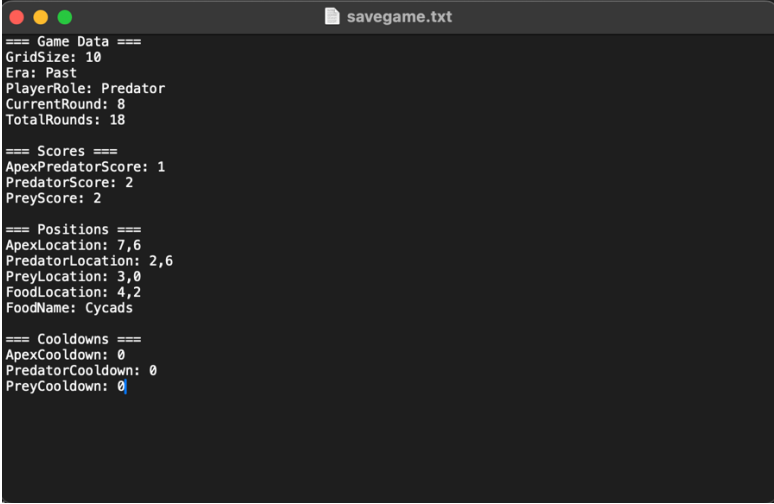
[Sun Jan 04 22:58:49 TRT 2026] [ERA: Present] [ROUND: 42/50] INFO: === Round 42 Ended ===
[Sun Jan 04 22:58:49 TRT 2026] [ERA: Present] [ROUND: 43/50] INFO: === Round 43 Started ===
[Sun Jan 04 22:58:49 TRT 2026] [ERA: Present] [ROUND: 43/50] MOVE: Polar Bear moved to: (3,4)
[Sun Jan 04 22:58:49 TRT 2026] [ERA: Present] [ROUND: 43/50] MOVE: Deer moved to: (1,8)
[Sun Jan 04 22:58:49 TRT 2026] [ERA: Present] [ROUND: 43/50] INFO: === Round 43 Ended ===
[Sun Jan 04 22:58:50 TRT 2026] [ERA: Present] [ROUND: 44/50] INFO: === Round 44 Started ===
[Sun Jan 04 22:58:50 TRT 2026] [ERA: Present] [ROUND: 44/50] MOVE: Polar Bear moved to: (2,5)
[Sun Jan 04 22:58:50 TRT 2026] [ERA: Present] [ROUND: 44/50] MOVE: Deer moved to: (2,8)
[Sun Jan 04 22:58:50 TRT 2026] [ERA: Present] [ROUND: 44/50] INFO: === Round 44 Ended ===
[Sun Jan 04 22:58:51 TRT 2026] [ERA: Present] [ROUND: 45/50] INFO: === Round 45 Started ===
[Sun Jan 04 22:58:51 TRT 2026] [ERA: Present] [ROUND: 45/50] MOVE: Polar Bear moved to: (3,6)
[Sun Jan 04 22:58:51 TRT 2026] [ERA: Present] [ROUND: 45/50] MOVE: Deer moved to: (0,9)
[Sun Jan 04 22:58:51 TRT 2026] [ERA: Present] [ROUND: 45/50] INFO: === Round 45 Ended ===
[Sun Jan 04 22:58:52 TRT 2026] [ERA: Present] [ROUND: 46/50] INFO: === Round 46 Started ===
[Sun Jan 04 22:58:52 TRT 2026] [ERA: Present] [ROUND: 46/50] MOVE: Polar Bear moved to: (2,7)
[Sun Jan 04 22:58:52 TRT 2026] [ERA: Present] [ROUND: 46/50] MOVE: Deer moved to: (1,8)
[Sun Jan 04 22:58:52 TRT 2026] [ERA: Present] [ROUND: 46/50] INFO: === Round 46 Ended ===
[Sun Jan 04 22:58:52 TRT 2026] [ERA: Present] [ROUND: 47/50] COLLISION: Predator ate Prey.
[Sun Jan 04 22:58:52 TRT 2026] [ERA: Present] [ROUND: 47/50] INFO: === Round 47 Started ===
[Sun Jan 04 22:58:52 TRT 2026] [ERA: Present] [ROUND: 47/50] MOVE: Polar Bear moved to: (1,8)
[Sun Jan 04 22:58:52 TRT 2026] [ERA: Present] [ROUND: 47/50] MOVE: Deer moved to: (2,5)
[Sun Jan 04 22:58:52 TRT 2026] [ERA: Present] [ROUND: 47/50] COLLISION: Apex Predator ate Predator.
[Sun Jan 04 22:58:52 TRT 2026] [ERA: Present] [ROUND: 47/50] INFO: === Round 47 Ended ===
[Sun Jan 04 22:58:54 TRT 2026] [ERA: Present] [ROUND: 48/50] INFO: === Round 48 Started ===
[Sun Jan 04 22:58:54 TRT 2026] [ERA: Present] [ROUND: 48/50] MOVE: Polar Bear moved to: (2,7)
[Sun Jan 04 22:58:54 TRT 2026] [ERA: Present] [ROUND: 48/50] MOVE: Deer moved to: (2,3)
[Sun Jan 04 22:58:54 TRT 2026] [ERA: Present] [ROUND: 48/50] INFO: === Round 48 Ended ===
[Sun Jan 04 22:58:55 TRT 2026] [ERA: Present] [ROUND: 49/50] INFO: === Round 49 Started ===

```

[Sun Jan 04 22:58:55 TRT 2026] [ERA: Present] [ROUND: 49/50] MOVE: Polar Bear
moved to: (2,6)
[Sun Jan 04 22:58:55 TRT 2026] [ERA: Present] [ROUND: 49/50] MOVE: Deer moved
to: (3,4)
[Sun Jan 04 22:58:55 TRT 2026] [ERA: Present] [ROUND: 49/50] INFO: === Round
49 Ended ===
[Sun Jan 04 22:58:56 TRT 2026] [ERA: Present] [ROUND: 50/50] INFO: === Round
50 Started ===
[Sun Jan 04 22:58:56 TRT 2026] [ERA: Present] [ROUND: 50/50] MOVE: Polar Bear
moved to: (3,5)
[Sun Jan 04 22:58:56 TRT 2026] [ERA: Present] [ROUND: 50/50] MOVE: Deer moved
to: (4,5)
[Sun Jan 04 22:58:56 TRT 2026] [ERA: Present] [ROUND: 50/50] INFO: === Round
50 Ended ===
[Sun Jan 04 22:58:56 TRT 2026] [ERA: Present] [ROUND: 50/50] STATUS: ---
SIMULATION FINISHED ---
[Sun Jan 04 22:58:56 TRT 2026] [ERA: Present] [ROUND: 50/50] WINNER: The
winner is: Wolf (Predator)
[Sun Jan 04 22:58:56 TRT 2026] [ERA: Present] [ROUND: 50/50] STATUS: --- GAME
OVER ---

```

4-) Start a new game, save the game and show the saved state in the text file: I started a game with 18 rounds on a 10x10 grid in the future era. After playing 8 rounds, I clicked the file icon in the top left corner and chose **Save & Exit**. Then the GUI was closed. When I started the game again, I selected **Load Game**, and the game continued from where it stopped. In the last screenshot, you can see that the game has ended.



```

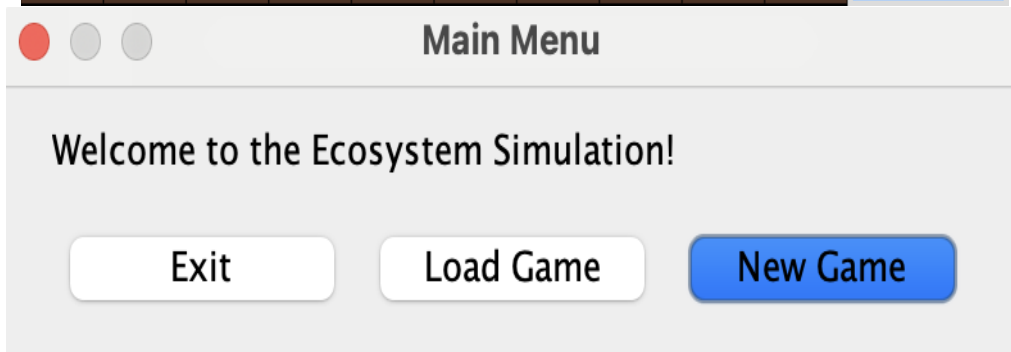
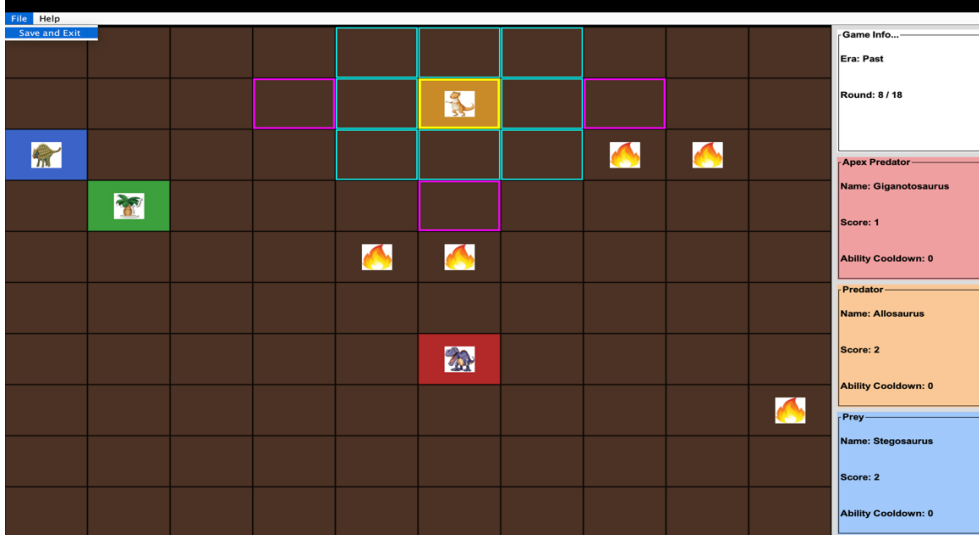
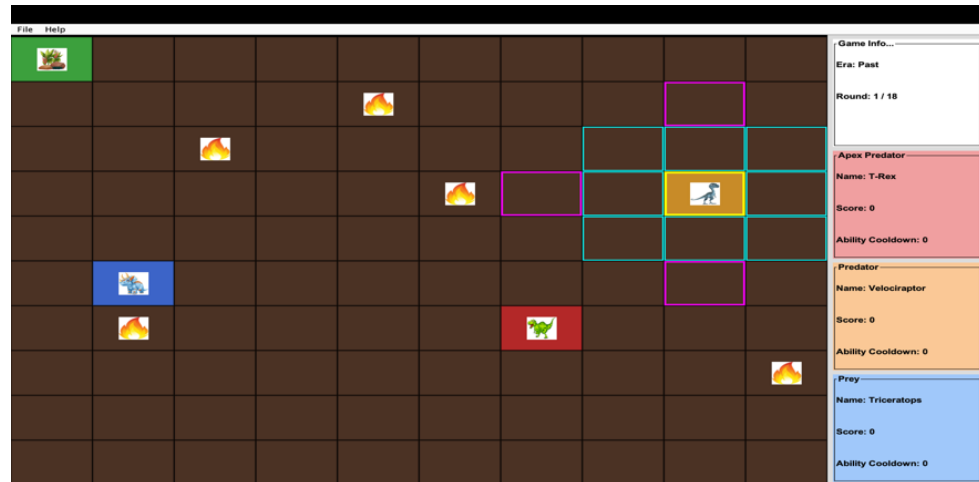
savegame.txt
=== Game Data ===
GridSize: 10
Era: Past
PlayerRole: Predator
CurrentRound: 8
TotalRounds: 18

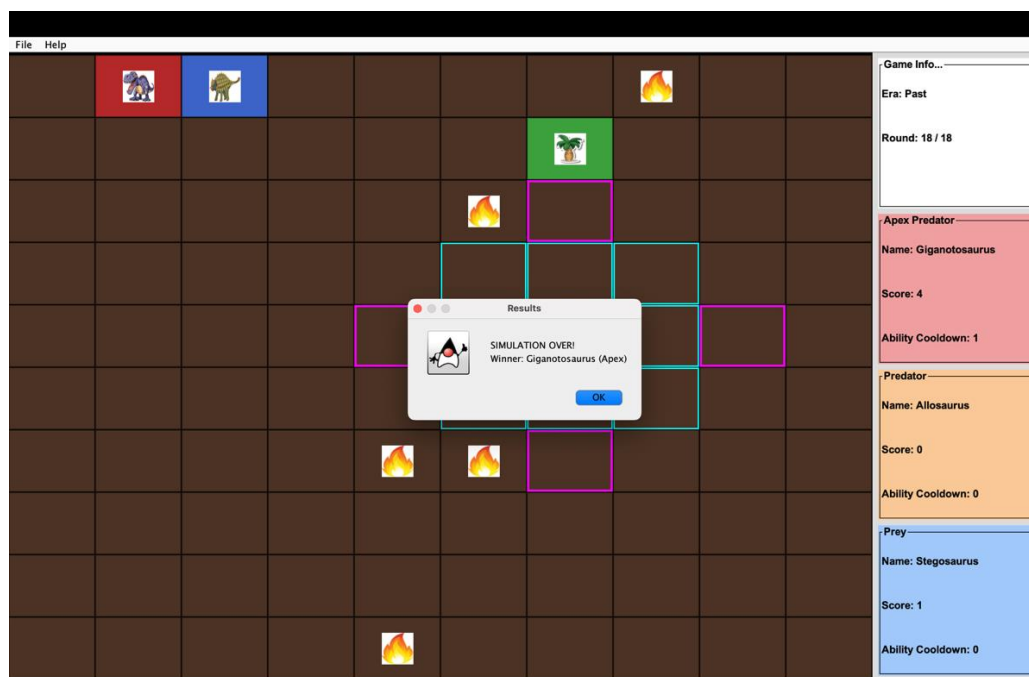
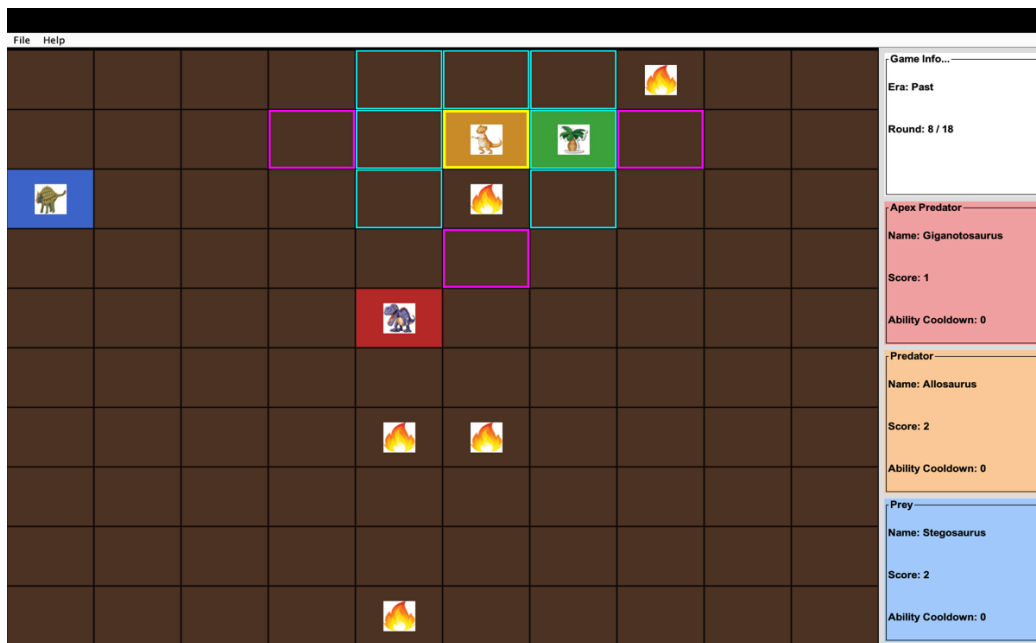
=== Scores ===
ApexPredatorScore: 1
PredatorScore: 2
PreyScore: 2

=== Positions ===
ApexLocation: 7,6
PredatorLocation: 2,6
PreyLocation: 3,0
FoodLocation: 4,2
FoodName: Cycads

=== Cooldowns ===
ApexCooldown: 0
PredatorCooldown: 0
PreyCooldown: 0

```





Mon Jan 05 00:05:04 TRT 2026] [ERA: Past] [ROUND: 1/18] INFO: === Round 1 Started ===

[Mon Jan 05 00:05:04 TRT 2026] [ERA: Past] [ROUND: 1/18] MOVE: T-Rex moved to: (1,6)

[Mon Jan 05 00:05:04 TRT 2026] [ERA: Past] [ROUND: 1/18] MOVE: Triceratops moved to: (5,6)

[Mon Jan 05 00:05:04 TRT 2026] [ERA: Past] [ROUND: 1/18] INFO: === Round 1 Ended ===

[Mon Jan 05 00:05:05 TRT 2026] [ERA: Past] [ROUND: 2/18] INFO: === Round 2 Started ===

[Mon Jan 05 00:05:05 TRT 2026] [ERA: Past] [ROUND: 2/18] MOVE: T-Rex moved to: (2,6)

[Mon Jan 05 00:05:05 TRT 2026] [ERA: Past] [ROUND: 2/18] MOVE: Triceratops moved to: (5,7)

[Mon Jan 05 00:05:05 TRT 2026] [ERA: Past] [ROUND: 2/18] COLLISION: Prey ate Food.

[Mon Jan 05 00:05:05 TRT 2026] [ERA: Past] [ROUND: 2/18] INFO: === Round 2 Ended ===

[Mon Jan 05 00:05:06 TRT 2026] [ERA: Past] [ROUND: 3/18] INFO: === Round 3 Started ===

[Mon Jan 05 00:05:06 TRT 2026] [ERA: Past] [ROUND: 3/18] MOVE: T-Rex moved to: (3,7)

[Mon Jan 05 00:05:06 TRT 2026] [ERA: Past] [ROUND: 3/18] MOVE: Triceratops moved to: (6,6)

[Mon Jan 05 00:05:06 TRT 2026] [ERA: Past] [ROUND: 3/18] INFO: === Round 3 Ended ===

[Mon Jan 05 00:05:07 TRT 2026] [ERA: Past] [ROUND: 4/18] INFO: === Round 4 Started ===

[Mon Jan 05 00:05:07 TRT 2026] [ERA: Past] [ROUND: 4/18] MOVE: T-Rex moved to: (4,6)

[Mon Jan 05 00:05:07 TRT 2026] [ERA: Past] [ROUND: 4/18] MOVE: Triceratops moved to: (8,7)

[Mon Jan 05 00:05:07 TRT 2026] [ERA: Past] [ROUND: 4/18] INFO: === Round 4 Ended ===

[Mon Jan 05 00:05:08 TRT 2026] [ERA: Past] [ROUND: 5/18] INFO: === Round 5 Started ===

[Mon Jan 05 00:05:08 TRT 2026] [ERA: Past] [ROUND: 5/18] MOVE: T-Rex moved to: (5,5)

[Mon Jan 05 00:05:08 TRT 2026] [ERA: Past] [ROUND: 5/18] MOVE: Triceratops moved to: (7,6)

[Mon Jan 05 00:05:08 TRT 2026] [ERA: Past] [ROUND: 5/18] INFO: === Round 5 Ended ===

[Mon Jan 05 00:05:09 TRT 2026] [ERA: Past] [ROUND: 6/18] INFO: === Round 6 Started ===

[Mon Jan 05 00:05:09 TRT 2026] [ERA: Past] [ROUND: 6/18] MOVE: T-Rex moved to: (6,6)

[Mon Jan 05 00:05:09 TRT 2026] [ERA: Past] [ROUND: 6/18] MOVE: Triceratops moved to: (6,5)

[Mon Jan 05 00:05:09 TRT 2026] [ERA: Past] [ROUND: 6/18] INFO: === Round 6 Ended ===

[Mon Jan 05 00:05:10 TRT 2026] [ERA: Past] [ROUND: 7/18] COLLISION: Predator ate Prey.

[Mon Jan 05 00:05:10 TRT 2026] [ERA: Past] [ROUND: 7/18] INFO: === Round 7 Started ===

[Mon Jan 05 00:05:10 TRT 2026] [ERA: Past] [ROUND: 7/18] MOVE: T-Rex moved to: (6,5)

[Mon Jan 05 00:05:10 TRT 2026] [ERA: Past] [ROUND: 7/18] MOVE: Triceratops moved to: (2,0)

[Mon Jan 05 00:05:10 TRT 2026] [ERA: Past] [ROUND: 7/18] COLLISION: Apex Predator ate Predator.

[Mon Jan 05 00:05:10 TRT 2026] [ERA: Past] [ROUND: 7/18] INFO: === Round 7 Ended ===

[Mon Jan 05 00:05:11 TRT 2026] [ERA: Past] [ROUND: 7/18] SAVE: === Game is saved to savegame.txt file ===

[Mon Jan 05 00:05:48 TRT 2026] [ERA: Past] [ROUND: 8/18] INFO: === Round 8 Started ===

[Mon Jan 05 00:05:48 TRT 2026] [ERA: Past] [ROUND: 8/18] MOVE: Giganotosaurus moved to: (3,3)

[Mon Jan 05 00:05:48 TRT 2026] [ERA: Past] [ROUND: 8/18] MOVE: Stegosaurus moved to: (4,0)

[Mon Jan 05 00:05:48 TRT 2026] [ERA: Past] [ROUND: 8/18] INFO: === Round 8 Ended ===

[Mon Jan 05 00:05:52 TRT 2026] [ERA: Past] [ROUND: 9/18] INFO: === Round 9 Started ===

[Mon Jan 05 00:05:52 TRT 2026] [ERA: Past] [ROUND: 9/18] MOVE: Giganotosaurus moved to: (2,2)

[Mon Jan 05 00:05:52 TRT 2026] [ERA: Past] [ROUND: 9/18] MOVE: Stegosaurus moved to: (3,1)

[Mon Jan 05 00:05:52 TRT 2026] [ERA: Past] [ROUND: 9/18] COLLISION: Apex Predator ate Predator.

[Mon Jan 05 00:05:52 TRT 2026] [ERA: Past] [ROUND: 9/18] INFO: === Round 9
Ended ===

[Mon Jan 05 00:05:53 TRT 2026] [ERA: Past] [ROUND: 10/18] INFO: === Round 10
Started ===

[Mon Jan 05 00:05:53 TRT 2026] [ERA: Past] [ROUND: 10/18] MOVE:
Giganotosaurus moved to: (3,1)

[Mon Jan 05 00:05:53 TRT 2026] [ERA: Past] [ROUND: 10/18] MOVE: Stegosaurus
moved to: (2,2)

[Mon Jan 05 00:05:53 TRT 2026] [ERA: Past] [ROUND: 10/18] INFO: === Round 10
Ended ===

[Mon Jan 05 00:05:54 TRT 2026] [ERA: Past] [ROUND: 11/18] INFO: === Round 11
Started ===

[Mon Jan 05 00:05:54 TRT 2026] [ERA: Past] [ROUND: 11/18] MOVE:
Giganotosaurus moved to: (2,2)

[Mon Jan 05 00:05:54 TRT 2026] [ERA: Past] [ROUND: 11/18] MOVE: Stegosaurus
moved to: (2,3)

[Mon Jan 05 00:05:54 TRT 2026] [ERA: Past] [ROUND: 11/18] INFO: === Round 11
Ended ===

[Mon Jan 05 00:05:54 TRT 2026] [ERA: Past] [ROUND: 12/18] INFO: === Round 12
Started ===

[Mon Jan 05 00:05:54 TRT 2026] [ERA: Past] [ROUND: 12/18] MOVE:
Giganotosaurus moved to: (2,3)

[Mon Jan 05 00:05:54 TRT 2026] [ERA: Past] [ROUND: 12/18] MOVE: Stegosaurus
moved to: (2,4)

[Mon Jan 05 00:05:54 TRT 2026] [ERA: Past] [ROUND: 12/18] INFO: === Round 12
Ended ===

[Mon Jan 05 00:05:55 TRT 2026] [ERA: Past] [ROUND: 13/18] INFO: === Round 13
Started ===

[Mon Jan 05 00:05:55 TRT 2026] [ERA: Past] [ROUND: 13/18] MOVE:
Giganotosaurus moved to: (2,4)

[Mon Jan 05 00:05:55 TRT 2026] [ERA: Past] [ROUND: 13/18] MOVE: Stegosaurus
moved to: (2,4)

[Mon Jan 05 00:05:55 TRT 2026] [ERA: Past] [ROUND: 13/18] COLLISION: Apex
Predator ate Prey.

[Mon Jan 05 00:05:55 TRT 2026] [ERA: Past] [ROUND: 13/18] INFO: === Round 13
Ended ===

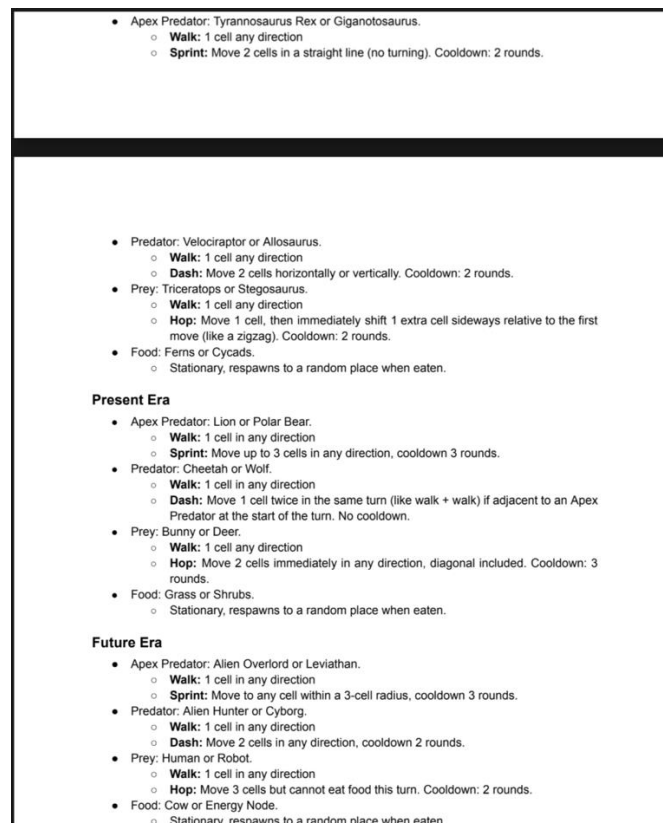
[Mon Jan 05 00:05:56 TRT 2026] [ERA: Past] [ROUND: 14/18] INFO: === Round 14
Started ===

[Mon Jan 05 00:05:56 TRT 2026] [ERA: Past] [ROUND: 14/18] MOVE:
Giganotosaurus moved to: (3,3)
[Mon Jan 05 00:05:56 TRT 2026] [ERA: Past] [ROUND: 14/18] MOVE: Stegosaurus
moved to: (0,0)
[Mon Jan 05 00:05:56 TRT 2026] [ERA: Past] [ROUND: 14/18] INFO: === Round 14
Ended ===
[Mon Jan 05 00:05:57 TRT 2026] [ERA: Past] [ROUND: 15/18] INFO: === Round 15
Started ===
[Mon Jan 05 00:05:57 TRT 2026] [ERA: Past] [ROUND: 15/18] MOVE:
Giganotosaurus moved to: (4,2)
[Mon Jan 05 00:05:57 TRT 2026] [ERA: Past] [ROUND: 15/18] MOVE: Stegosaurus
moved to: (1,1)
[Mon Jan 05 00:05:57 TRT 2026] [ERA: Past] [ROUND: 15/18] INFO: === Round 15
Ended ===
[Mon Jan 05 00:05:57 TRT 2026] [ERA: Past] [ROUND: 16/18] INFO: === Round 16
Started ===
[Mon Jan 05 00:05:57 TRT 2026] [ERA: Past] [ROUND: 16/18] MOVE:
Giganotosaurus moved to: (3,1)
[Mon Jan 05 00:05:57 TRT 2026] [ERA: Past] [ROUND: 16/18] MOVE: Stegosaurus
moved to: (0,1)
[Mon Jan 05 00:05:57 TRT 2026] [ERA: Past] [ROUND: 16/18] COLLISION: Apex
Predator ate Predator.
[Mon Jan 05 00:05:57 TRT 2026] [ERA: Past] [ROUND: 16/18] INFO: === Round 16
Ended ===
[Mon Jan 05 00:05:59 TRT 2026] [ERA: Past] [ROUND: 17/18] INFO: === Round 17
Started ===
[Mon Jan 05 00:05:59 TRT 2026] [ERA: Past] [ROUND: 17/18] MOVE:
Giganotosaurus moved to: (2,1)
[Mon Jan 05 00:05:59 TRT 2026] [ERA: Past] [ROUND: 17/18] MOVE: Stegosaurus
moved to: (0,1)
[Mon Jan 05 00:05:59 TRT 2026] [ERA: Past] [ROUND: 17/18] INFO: === Round 17
Ended ===
[Mon Jan 05 00:06:00 TRT 2026] [ERA: Past] [ROUND: 18/18] INFO: === Round 18
Started ===
[Mon Jan 05 00:06:00 TRT 2026] [ERA: Past] [ROUND: 18/18] MOVE:
Giganotosaurus moved to: (0,1)
[Mon Jan 05 00:06:00 TRT 2026] [ERA: Past] [ROUND: 18/18] MOVE: Stegosaurus
moved to: (0,2)

```
[Mon Jan 05 00:06:00 TRT 2026] [ERA: Past] [ROUND: 18/18] INFO: === Round 18
Ended ===
[Mon Jan 05 00:06:00 TRT 2026] [ERA: Past] [ROUND: 18/18] STATUS: ---
SIMULATION FINISHED ---
[Mon Jan 05 00:06:00 TRT 2026] [ERA: Past] [ROUND: 18/18] WINNER: The winner
is: Giganotosaurus (Apex)
[Mon Jan 05 00:06:00 TRT 2026] [ERA: Past] [ROUND: 18/18] STATUS: --- GAME
OVER ---
```

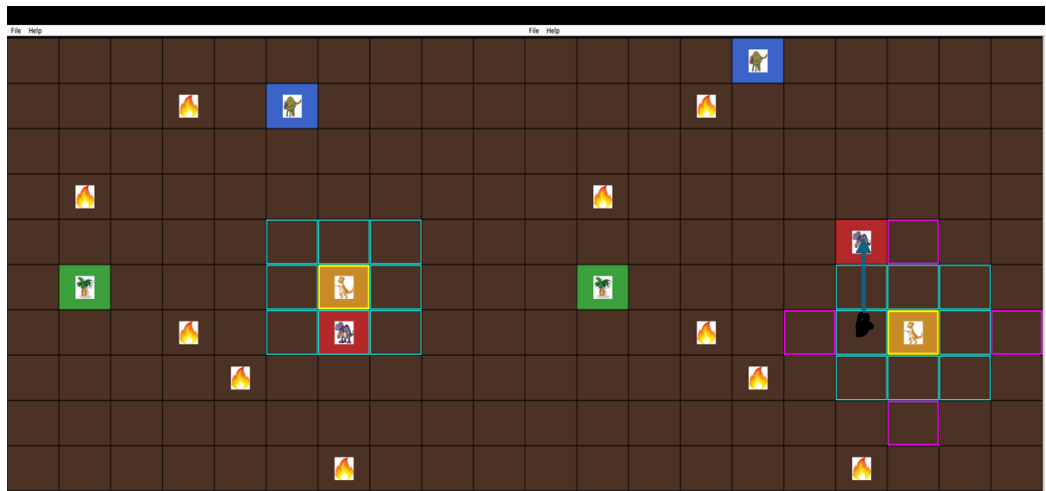
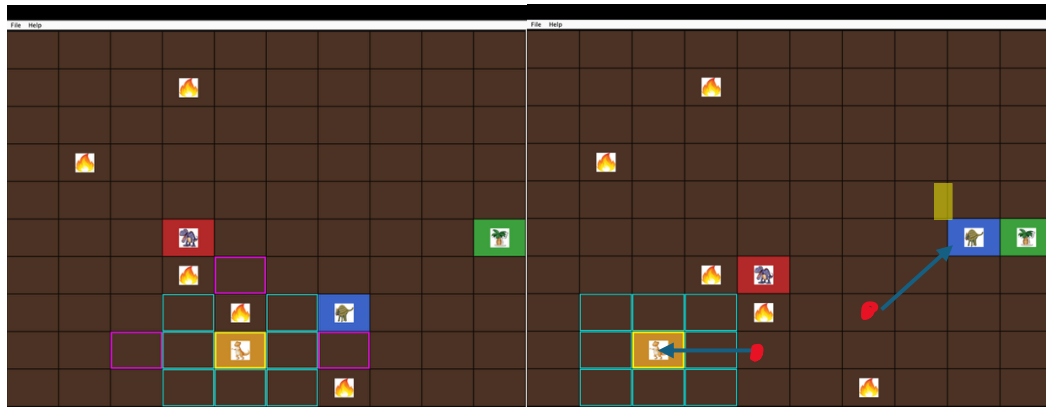
5-) Play a new game and demonstrate the difference in special moves, and animal icons:

I put a screenshot from movement summary for characters. I will demonstrate by screenshots from my game.

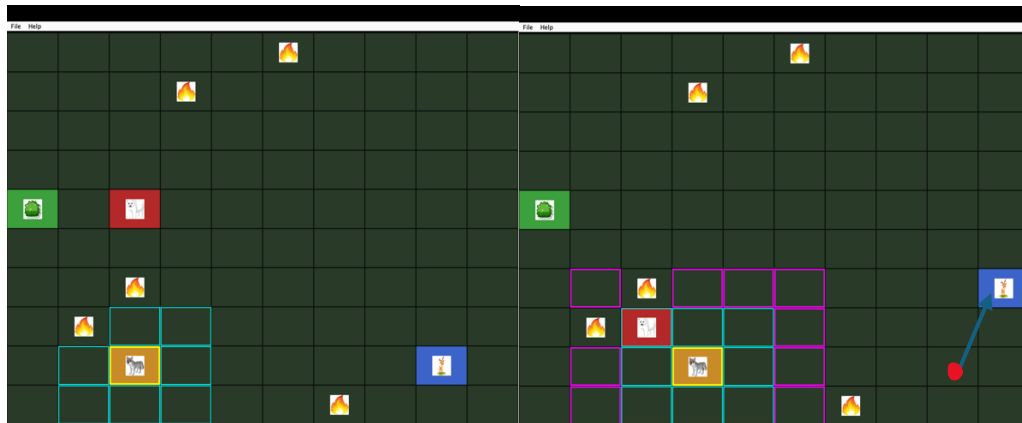


a-) Past Era:

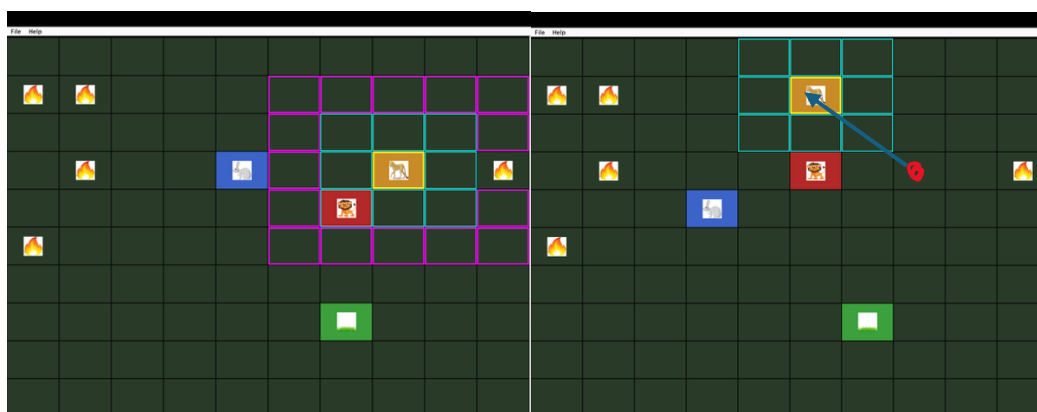
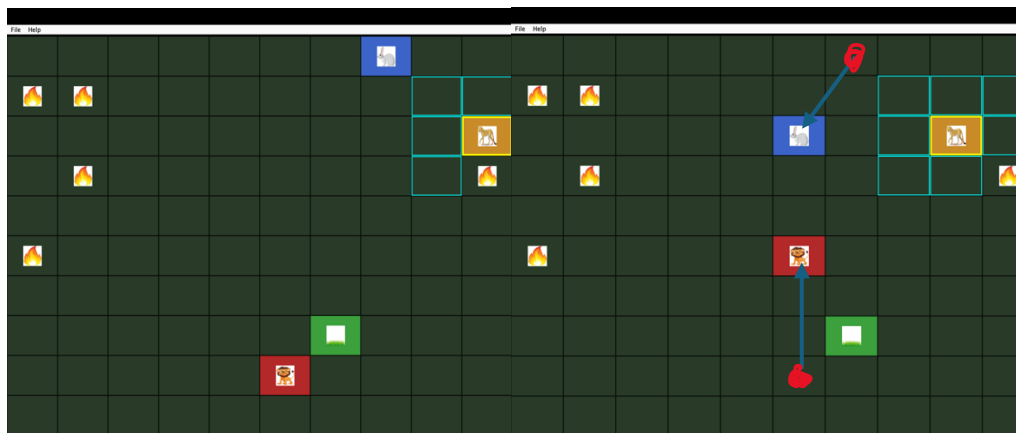
i-) FoodChain 2:



ii-) FoodChain1:

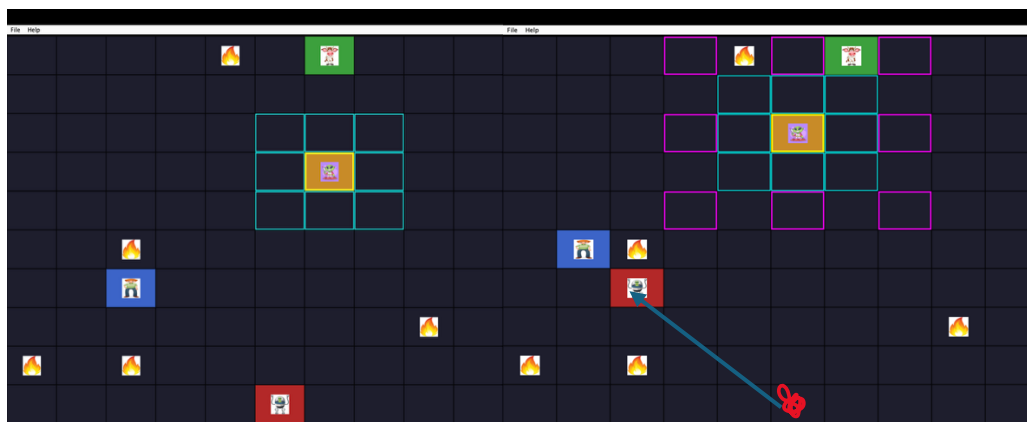
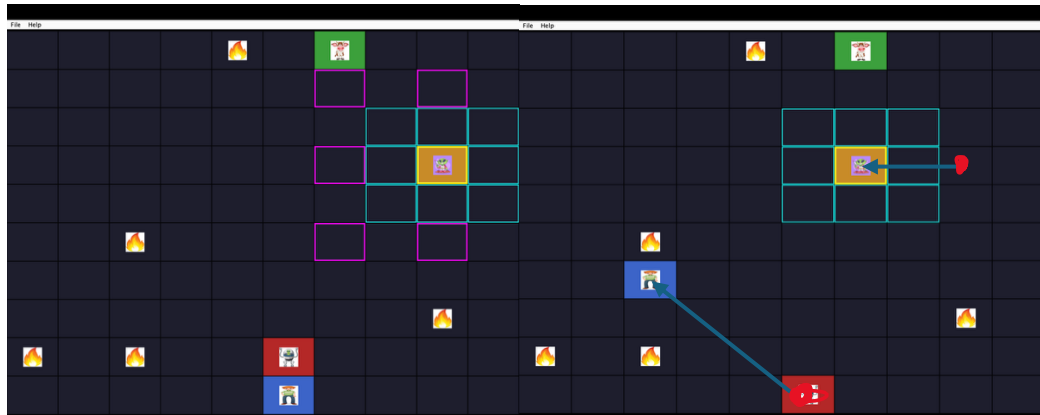


ii-) FoodChain 1:

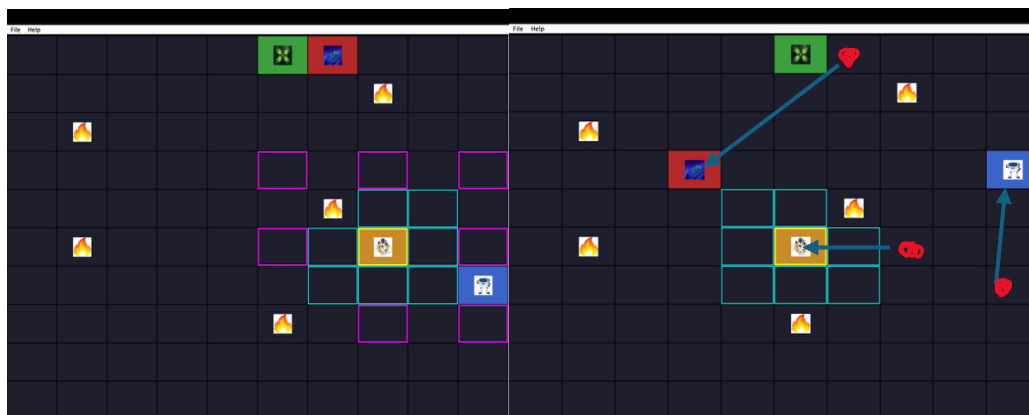


c-) Future era:

i-) foodChain 1-):



ii-) FoodChain2:



PART 2:

a-)Packages, classes and methods descriptions:

1. characters Package

This package serves as the logical core for all characters within the ecosystem. It determines how they maintain their status, manage their special powers, and interact with the grid environment.

1.1 Characters Class (Abstract): The `Characters` class is the blueprint for all living units. It establishes the fundamental state variables and common behaviors required for survival.

- **updateCooldown():** This method is called during every round transition. It checks the current `cooldown` value of the character. If the value is greater than zero, it decreases the integer by 1, slowly allowing the character to use of its special ability again.
- **canUseAbility():** This method performs a logical check to see if the `cooldown` variable is exactly zero. If the result is true, it signals to the `GameManager` that the character is ready to perform a complex move like a Sprint or Dash.
- **moveTo(int x, int y):** This abstract method requires every subclass to define how it updates its location variables. It changes the internal `locationX` and `locationY` to match the target grid coordinates. It is inherited to `prey`, `predator` and `apexPredator` class.

1.2 ApexPredator Class: This class represents the most powerful hunters like polar bear.

- **specialAbility():** This method handles the logic for the **Sprint** action. It calculates a movement of 2 or 3 cells in a straight line or radius, depending on the current era. It then sets the character's cooldown to a high value (usually 2 or 3 rounds) to prevent immediate reuse.

1.3 Predator Class: The `Predator` class defines intermediate hunters like Velociraptors or Wolves.

- **specialAbility():** It implements the **Dash** ability. In the **Past Era**, it checks if the move is strictly horizontal or vertical. In the **Present Era**, it includes a special conditional check to see if an `ApexPredator` is in an adjacent cell before allowing a double move.

1.4 Prey Class: This class represents the characters that must avoid capture to survive.

- **specialAbility():** This method manages the **Hop** ability. In the **Past Era**, it validates a "zigzag" pattern where the character moves 2 cells in one direction and 1 cell sideways. This logic is designed to help the Prey break the line of sight of straight-moving predators.

1.5-1.6 Walkable Interface and moveException: These components ensure the simulation remains stable and follows a strict contract.

- **Walkable Interface:** It defines the method signatures for `moveTo` and `specialAbility`, ensuring that any new character added to the game will have a predictable movement structure.
- **moveException Class:** This class extends the standard Java `Exception`. When a character attempts to move into a fire obstacle or out of the grid boundaries, this class generates a custom error message to stop the illegal action.

2. Computer Package

This package handles the Artificial Intelligence (AI) logic for the simulation. It determines how computer-controlled characters, such as the Apex Predator and Prey, navigate the grid and interact with their surroundings.

2.1 AIManager Class: This class serves as the executor for the AI-controlled turns. It manages the movement of non-player characters based on the specific rules of the current era.

- **handleApexTurn():** This method manages the turn for the Apex Predator. It first calls the `getClosest` method to identify the nearest available target, such as a Prey or a Predator. It then checks if the character `canUseAbility()` to determine the movement

range from `EraSettings`. If the era is **Past** and an ability is used, the method ensures the character moves toward the target using the `approach` logic.

- **handlePreyTurn():** This method manages the turn for the Prey by evaluating its environment to find food while avoiding predators. It determines the allowed movement range and then requests the optimal coordinates from the `FindBestDecision` class.
- **moveTo(Characters c, int x, int y, Ecosystem eco):** A utility method that ensures AI characters do not move into obstacles or out of the grid boundaries. It uses `Math.max` and `Math.min` to keep the `x` and `y` values within the `gridSize` and performs an `isObstacle()` check before updating the character's location.
- **approach(Characters c, int tx, int ty, Ecosystem eco):** Specifically handles the movement logic for the **Past** era. It allows the Apex Predator to move in a straight line toward a target by incrementing or decrementing its coordinates based on the target's relative position.
- **Relationships:** The `AIManager` acts as a controller that uses the `FindBestDecision` class to obtain the optimal coordinates for each turn, effectively separating the execution of movement from the decision-making process.

2.2 FindBestDecision Class: This class contains the mathematical algorithms and scoring systems used to determine the best possible move for an AI character.

- **getPreyMove():** This method scans all cells within the current movement range of the Prey. It identifies the nearest threat and the nearest food to evaluate every possible destination. It calculates a "Position Score" for each candidate cell and selects the one with the highest value.
- **getApexMove():** This method evaluates cells within the character's range to find the position that minimizes the distance to the nearest target. It loops through the grid using `getDistance` and `isObstacle` boundaries and uses the `getDistance` helper to find the most efficient hunting path.
- **Scoring Logic:** The class uses a specific formula to prioritize survival: $\text{double positionScore} = (\text{threatDist} * 1.5) - \text{foodDist};$. By multiplying the threat distance by 1.5, the AI ensures that escaping from an enemy is more important than reaching a food source.

- **Distance Metrics:** It calculates distances between grid points using a logic that considers the maximum difference between `x` and `y` coordinates, often referred to as distance $(\text{Max}(|x_1 - x_2|, |y_1 - y_2|))$. (I used that formula here. If I used normal euclidean formula it calculates the diagonals as much more than normal movement. However in definition of walk movement they are all equal.)
- **Relationships:** It provides static decision-making services to the `AIManager`, allowing for centralized updates to the AI's intelligence without modifying the movement execution code.

3. gamePlaces Package

This package handles the spatial logic of the simulation. It defines the environment where characters interact, manages obstacles, and controls the lifecycle of food resources.

3.1 Ecosystem Class Methods: The `Ecosystem` class serves as the high-level manager for the coordinate system, handling the initialization and reset of the environment.

- **Ecosystem(int gridSize):** This constructor initializes the 2D array that represents the simulation world. It uses nested loops to populate every index of the array with a new `GridCell` object. This ensures that the grid is fully instantiated and ready for object placement from the start of the game.
- **generateObstacles(int obstacleCount):** This method uses a `while` loop to place a specific number of obstacles randomly across the grid. It utilizes the `SecureRandom` class to select random `x` and `y` coordinates. Before marking a cell as an obstacle, it verifies that the cell is `totallyEmpty()` to prevent overlapping with existing characters or resources.
- **placeCharacter(Character character, int x, int y):** This method acts as a safety gate for character placement. It first checks if the target `x` and `y` coordinates are within the grid boundaries. It also ensures the destination is not an obstacle before updating the character reference in the `GridCell`.
- **placeFood(Food food, int x, int y):** Similar to character placement, this method assigns a `Food` object to a specific grid coordinate. This allows Prey units to identify and consume resources at that specific location.

- **respawn(Object character):** This is a complex logic method that handles the relocation of entities when they are eaten or consumed. It first scans the grid to create a `List` of all available coordinates that are not obstacles and are currently empty. It then randomly selects one coordinate from this list and updates the internal position of the passed object, whether it is a `Characters` or a `Food` instance.
- **clearCharacters() and clearFood():** These utility methods iterate through the entire 2D array using nested `for` loops to set all character or food references to `null`. This process is essential for the `refreshBoard()` function in the `GameManager` to ensure that old positions are removed before drawing the new positions for a new round.

3.2 Location Class Methods: The `Location` class defines the atomic data structure of the grid, representing what exists at a single point in space.

- **totallyEmpty():** This is the primary logical check for the class. It returns a boolean value based on the condition that the `characters` reference is `null`, the `food` reference is `null`, and the `obstacle` flag is `false`. This method is vital for the `AI` and `Ecosystem` class to find valid movement or respawn paths.

3.3 Food Class Methods: The `Food` class is a simple data object that stores the state of stationary resources.

- **Food(String name, int x, int y):** The constructor assigns a thematic name, selected via the `Era` class, and initial coordinates to the resource. This name changes based on the era, such as "Ferns" in the Past or "Energy Node" in the Future.

4. modesAndEcosystems Package

4.1 Era Class Methods: The `Era` class provides utility functions to maintain thematic consistency by selecting era-appropriate names and resources.

- **pickCharacterName(String era):** This method acts as a bridge to the `FileHandler`. it retrieves a list of potential names from external files (e.g., `future_animals.txt`). It uses `SecureRandom` (as asked in pdf I used `SecureRandom` instead `random` or etc.) to

ensure a non-deterministic selection. To prevent system crashes, it includes a safety check that returns a "Default" string if the name list is empty.

- **pickFoodName(String era):** This method uses a logic branch to assign food names based on the active era. It uses a boolean randomizer to choose between two options per era, such as "Ferns" or "Cycads" in the **Past**, and "Cow" or "Energy Node" in the **Future**.

4.2 EraSettings Class Methods: The `EraSettings` class provides the static data used to balance the game's mechanics.

- **getAbilityCooldown(String animalType, String era):** This method returns the required waiting period after using a special ability. In the **Past**, it returns 2 rounds for all. In the **Present**, it returns 0 for the Predator to allow consecutive dashes near an Apex, but 3 for others. In the **Future**, it returns 3 for Apex and 2 for Predator/Prey.
- **getAbilityDistance(String animalType, String era):** This method defines the maximum reach of an ability. For example, in the **Future**, it returns 2 for Predators to limit their dash range, while returning 3 for Apex and Prey units to maintain the survival balance.

4.3 GameManager Class Methods: The `GameManager` methods represent the primary engine that coordinates character interaction and the game loop.

- **runRound():** This is the central loop of the simulation. It begins by logging the round start, then triggers the AI turns for Apex Predators and Preys. After movement, it calls `checkCollisions()` and `refreshBoard()` to update the UI. Finally, it checks if the game has ended based on the `totalRounds` limit.
- **movePlayer(int targetX, int targetY):** This method manages the human player's input. It first calculates the distance of the move. If the move is a special ability (`distance > 1`), it verifies the move's legality and sets the appropriate cooldown using `EraSettings`. It ensures that `updateAllCooldowns()` is called so that time progresses for all characters.
- **isMoveLegal(Characters p, int dx, int dy, int dist):** This is the geometric validation method.
 - **Past Predator:** It ensures the move is a straight line by checking if either `dx` or `dy` is zero.

- **Future Predator:** It prevents "L-shaped" moves by allowing only straight lines or perfect diagonals where $(dx = 0 \text{ or } dy = 0)$ or a perfect diagonal $(dx = dy)$.
 - **Prey Past:** It enforces the "zigzag" rule by requiring a coordinate shift of exactly or .
- **updateAllCooldowns():** A private utility method that iterates through every character list and calls `updateCooldown()` to decrease their waiting timers by 1.
- **checkCollisions():** This method identifies overlapping coordinates. It awards **1 point** to an Apex Predator for catching any target, and **3 points** to a Predator for catching a Prey. It then triggers the `ecosystem.respawn()` method for the consumed unit.
- **handlePlayerCollisions(Predator player):** A specific handler for the human player that rewards **3 points** when the player lands on a Prey's square and records the event in the log.
- **refreshBoard():** This method clears the old character and food references from the `Ecosystem` grid and replaces them according to their new coordinates, ensuring the visual representation is accurate.
- **loadData():** Executed during initialization, this method parses the specifically era character files, creates the character objects, and performs the initial random placement on the board.
- **getWinner():** This method calculates the final scores and returns a string identifying whether the Apex, Predator, or Prey had the highest performance at the end of the rounds.
- **isNearApex(Characters p):** Approximity check used in the **Present Era** to determine if a Predator is close enough to an Apex Predator to trigger its special dash.
- **savedData(FileHandler fh):** This method restores the game state from a `FileHandler` object, allowing players to resume a session with their previous scores, rounds, and positions intact.

5. FileIO Package

5.1 FileHandler Class Methods: This package is responsible for all permanent data operations. It ensures that the simulation can interact with the computer's file system to retrieve character names, record movement logs, and preserve game progress.

- **getCharacterNames(String selectedEra):** This method dynamically constructs a filename based on the current era, such as `past_animals.txt`, and reads it using a **Scanner**. It searches for lines containing a colon to identify "Food Chain" entries. Then, it splits and trims the string to return a **List<String>** of names like "Tyrannosaurus Rex" or "Allosaurus" for use in the simulation.
- **recordLog(String status, String message):** This is the primary logging method for the system. It opens `log.txt` in "append mode" using **FileWriter(LOG_PATH, true)**, which ensures that new information is added to the end of the file instead of overwriting existing data. Each entry is automatically timestamped using the **java.util.Date**^[11] class to provide a chronological history of all game events.
- **exportFile(String fileName):** This method handles the "Save Game" functionality by serializing the current state of the ecosystem into a structured text format. To make the file human-readable, it converts internal zero-based coordinates to 1 indexed positions by adding 1. Since in loops like iteration starts from 0. It also includes a logical check to ensure that a **FoodName** is only written to the file if it is not null or empty().
- **importFile(String fileName):** This method provides the "Load Game" functionality. It parses a saved text file line by line to identify specific keys, such as **ApexLocation**, and their corresponding values. It reverses the coordinate conversion by subtracting 1 from the saved positions to restore the internal grid logic used by the simulation.

6. View Package

The View package is the visual part of the simulation. Its job is to show the ecosystem on the screen and send user actions (like mouse clicks) to the game logic. It uses a modular structure where different classes handle specific tasks like loading images, managing the statistics panel, or controlling the grid cells.

6.1 GameWindow Class Methods: This class is the main window of the application. It holds the grid, the menu, and the statistics panel together.

- **GameWindow(GameManager manager, int size):** This is the constructor. It sets the title of the window to "Ecosystem Simulation - Adnan Gültekin". It also creates the menu

bar, sets up the grid board with `initializeBoard`, and adds the `InfoBox` panel to the right side.

- **`initializeBoard(int size)`:** This method creates a main panel with a black background. It uses a `GridLayout` to arrange every cell correctly according to the grid size.
- **`setupCell(int r, int c)`:** This method creates a new `GridCell` for specific coordinates. It also attaches a `CellController` to the cell so the game can detect when a user clicks on it.
- **`updateDisplay()`:** This is^[1] the most important visual method. It checks every square in the ecosystem grid. It looks for characters, food, or obstacles and updates the background colors and icons. Finally, it calls `infoPanel.updateAllStats` to keep all numbers and scores updated.
- **`updateCellBorders(int i, int j, Predator player, String era, boolean isObs)`:** This method^[2] gives visual help to the player. It calculates the distance between the player and other squares. It paints **Yellow** borders for the player's current spot, **Cyan** for normal walk areas, and **Magenta** for special ability areas.
- **`setupMenuBar()`:** This creates the top menu. It adds a "File" menu for saving and an "Instructions" menu to show the help screen.
- **`getEraColor(String era)`:** This returns a specific color depending on the era. For example, it returns brown for **Past**, dark green for **Present**, and dark blue for **Future**.
- **`performIO()`:** This^[3] handles saving the game. It collects all scores, locations, and cooldowns from the `GameManager` and gives them to the `FileHandler`. It also converts internal 0-based positions to 1-based positions for the text file before closing the game.

6.2 GridCell Class Methods: Each `GridCell` represents one square on the map and manages its own images.

- **`setOccupant(Characters c, Food f, boolean isObstacle, Color bgColor)`:** This^[4] is the main method to update the cell's look. If there is an obstacle, it shows "fire.png". If there is a character, it finds the correct ".png" file using the character's name.

- **getCachedIcon(String filename):** This^[5] method manages images efficiently. If an image was loaded before, it takes it from the cache. Otherwise, it loads the file, scales it to 45x45 pixels, and saves it in the `iconCache`.
- **totallyEmpty():** This returns true only if the square has no character, no food, and no obstacle.

6.3 InfoBox Class Methods: This class manages the data panel on the right side of the screen.

- **initializeSectionPanel(String sectionTitle, Color backgroundColor):** This creates^[6] titled sections with different colors for the Apex, Predator, and Prey.
- **updateAllStats():** This method^[7] receives updated data from the `GameManager` and changes the text on all labels. This ensures the player always sees the correct scores and cooldown times.

6.4 CellController Class Methods: This class connects user clicks to the game logic.

- **mousePressed(MouseEvent e):** This detects left-clicks^[8]. It sends the coordinates to `manager.movePlayer`. If the move is valid, it refreshes the screen and checks if someone won the game.

6.5 PhotoLoader Class Methods: This is a utility class for fast image loading.

- **getImage(String fileName):** This method uses a `HashMap` to store icons in memory^[9]. This stops the computer from reading the hard drive too many times, making the game run faster.

6.6 StartScreen Class Methods: This class handles the first menus and settings.

- **showGameInfo():** This shows a pop-up window with instructions about roles and movement colors.

- **showSetup():** This^[10] creates a form to choose grid size, era, and rounds. It uses a loop to check for errors; for example, if the user enters less than 10 rounds, it shows an error message.

7. Main Class

The **main** method is the starting point of the simulation. It follows a simple set of steps to get the game ready based on what the user wants to do:

- **Getting Setup Info:** The program first opens the setup screen to ask the user for game settings like grid size and rounds.
- **Checking for Cancel:** If the user closes the setup window without entering anything, the program stops immediately to prevent any crashes.
- **Choosing the Mode:** The code looks at the first piece of info from the setup to see if the user wants to load an old game or start a brand new one.
- **Loading an Old Game:** If the user chooses "load" the FileHandler reads the savegame.txt file to restore the previous scores and character positions.
- **Starting a New Game:** If it is a new game, the program uses the chosen size and era to create a fresh Ecosystem and GameManager.
- **Showing the Game:** Once everything is set up, the GameWindow opens to finally show the simulation on the screen.

b-) .txt file processing details

b-1) log.txt:

```
[Mon Jan 05 23:34:56 TRT 2026] [ERA: Past] [ROUND: 1/15] INFO: === Round 1
Started ===
```

```
[Mon Jan 05 23:34:57 TRT 2026] [ERA: Past] [ROUND: 2/15] INFO: === Round 2
Started ===
```

```
[Mon Jan 05 23:34:57 TRT 2026] [ERA: Past] [ROUND: 2/15] COLLISION: Apex
Predator ate Predator.
```

```
[Mon Jan 05 23:34:58 TRT 2026] [ERA: Past] [ROUND: 3/15] INFO: === Round 3
Started ===
```

[Mon Jan 05 23:34:58 TRT 2026] [ERA: Past] [ROUND: 4/15] INFO: === Round 4
Started ===

[Mon Jan 05 23:34:58 TRT 2026] [ERA: Past] [ROUND: 4/15] COLLISION: Prey ate
Food.

[Mon Jan 05 23:35:00 TRT 2026] [ERA: Past] [ROUND: 5/15] INFO: === Round 5
Started ===

[Mon Jan 05 23:35:01 TRT 2026] [ERA: Past] [ROUND: 6/15] INFO: === Round 6
Started ===

[Mon Jan 05 23:35:02 TRT 2026] [ERA: Past] [ROUND: 7/15] INFO: === Round 7
Started ===

[Mon Jan 05 23:35:03 TRT 2026] [ERA: Past] [ROUND: 8/15] INFO: === Round 8
Started ===

[Mon Jan 05 23:35:03 TRT 2026] [ERA: Past] [ROUND: 8/15] COLLISION: Prey ate
Food.

[Mon Jan 05 23:35:05 TRT 2026] [ERA: Past] [ROUND: 8/15] STATUS: game is
saved

[Mon Jan 05 23:36:11 TRT 2026] [ERA: Past] [ROUND: 9/15] INFO: === Round 9
Started ===

[Mon Jan 05 23:36:12 TRT 2026] [ERA: Past] [ROUND: 10/15] INFO: === Round 10
Started ===

[Mon Jan 05 23:36:13 TRT 2026] [ERA: Past] [ROUND: 11/15] INFO: === Round 11
Started ===

[Mon Jan 05 23:36:14 TRT 2026] [ERA: Past] [ROUND: 12/15] INFO: === Round 12
Started ===

[Mon Jan 05 23:36:14 TRT 2026] [ERA: Past] [ROUND: 13/15] INFO: === Round 13
Started ===

[Mon Jan 05 23:36:15 TRT 2026] [ERA: Past] [ROUND: 14/15] INFO: === Round 14
Started ===

[Mon Jan 05 23:36:15 TRT 2026] [ERA: Past] [ROUND: 14/15] COLLISION: Apex
Predator ate Predator.

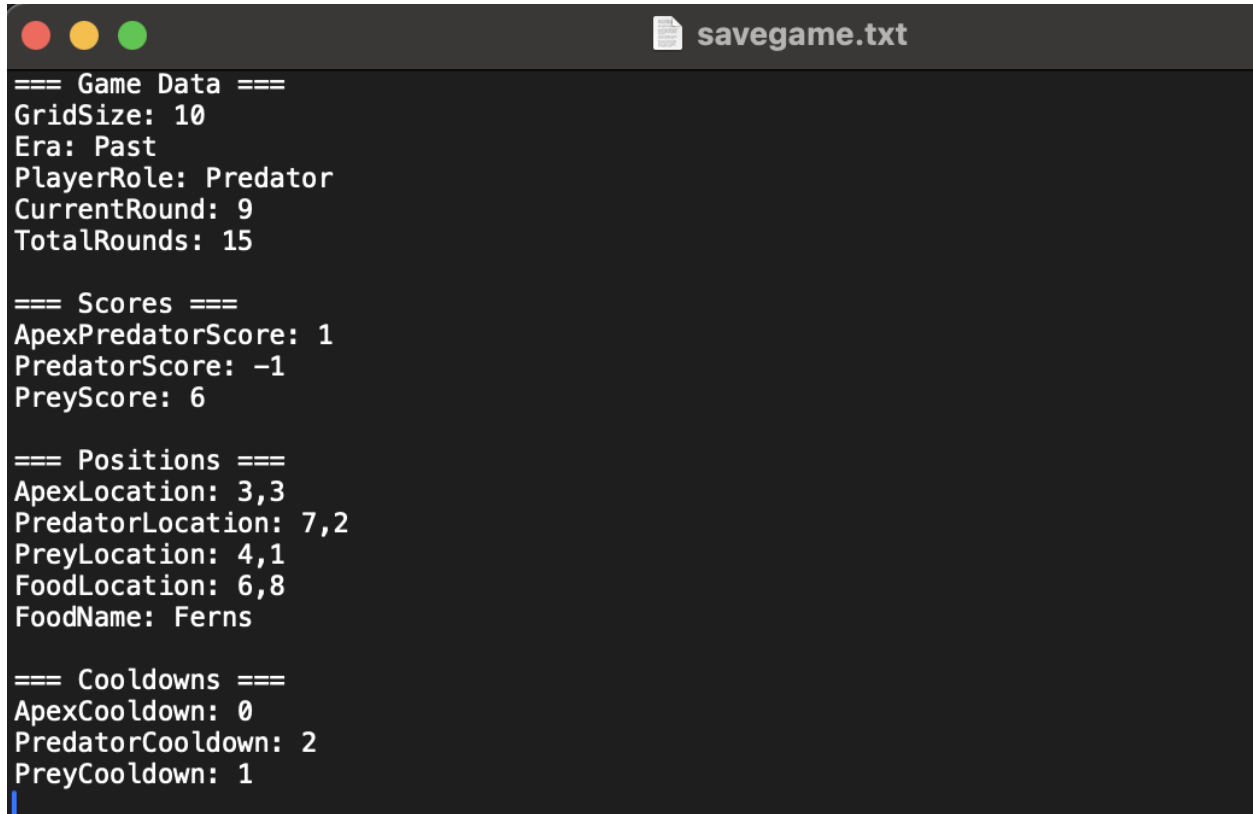
[Mon Jan 05 23:36:16 TRT 2026] [ERA: Past] [ROUND: 15/15] INFO: === Round 15
Started ===

[Mon Jan 05 23:36:16 TRT 2026] [ERA: Past] [ROUND: 15/15] STATUS: ---
SIMULATION FINISHED ---

[Mon Jan 05 23:36:16 TRT 2026] [ERA: Past] [ROUND: 15/15] WINNER: The winner
is: Stegosaurus (Prey)

[Mon Jan 05 23:36:16 TRT 2026] [ERA: Past] [ROUND: 15/15] STATUS: --- GAME OVER ---

b-2) savegame.txt



```
=== Game Data ===
GridSize: 10
Era: Past
PlayerRole: Predator
CurrentRound: 9
TotalRounds: 15

=== Scores ===
ApexPredatorScore: 1
PredatorScore: -1
PreyScore: 6

=== Positions ===
ApexLocation: 3,3
PredatorLocation: 7,2
PreyLocation: 4,1
FoodLocation: 6,8
FoodName: Ferns

=== Cooldowns ===
ApexCooldown: 0
PredatorCooldown: 2
PreyCooldown: 1
```

b-3) I also have .png files to get the images of characters’.

c-)Additional design details:

The interface uses a hierarchical structure where basic components like **JLabels** and **JPanels** are nested inside larger containers to create an organized, interactive layout.

c-1) GameWindow

The `GameWindow` class extends `JFrame` and serves as the primary window for the application.

- **Method: initializeBoard:** This method creates a central `JPanel` called `board`. It uses a `GridLayout` to organize the board into a square grid based on the size chosen by the user. The background is set to black to create a "grid line" effect between cells.
- **Method: setupMenuBar:** This method builds a `JMenuBar` at the top of the window. It uses `JMenu` and `JMenuItem` to provide "Save and Exit" and "Instructions" functions.
- **Layout Management:** The window uses `BorderLayout`. The grid is placed in the `CENTER` for maximum visibility, while the statistics panel is placed in the `EAST` (right side).

c-2) GridCell

Each `GridCell` is a `JPanel` representing one coordinate on the map.

- **Method: initComponents:** This method creates a `JLabel` to hold icons for characters, food, or obstacles. The label is non-opaque (`setOpaque(false)`) so the cell's background color remains visible behind the image.
- **Method: setupLayout:** The cell uses a `BorderLayout` and places the icon label in the `CENTER` to ensure the entity image is perfectly centered within the square.
- **Border Highlighting:** The cell uses `BorderFactory.createLineBorder` to change its appearance. This is used to display yellow, cyan, and magenta borders for player movement guidance.

c-3) InfoBox

The `InfoBox` is a `JPanel` that functions as a vertical statistics container.

- **Main Container Logic:** It uses a `BoxLayout` along the Y-axis, allowing sub-panels to be stacked vertically.
- **Method: initializeSectionPanel:** This creates specialized `JPanel` objects for each character role.

It uses a `GridLayout(3, 1)` so that Name, Score, and Cooldown labels are aligned in three rows.

It applies a `TitledBorder` to provide a visible frame and a title (like apex predator) for each section.

- **Method: `setupLabelFonts`:** This method loops through all `JLabel` objects and applies a consistent bold font to ensure visual uniformity across the GUI.

c-4) CellController

The `CellController` extends `MouseAdapter` to handle physical user inputs.

- **Method: `mousePressed`:** This method is the primary input handler. While it is not a visual component itself, it is attached to every `GridCell` (`JPanel`) using `addMouseListener`. This allows the GUI to detect when a specific square is clicked and trigger the movement logic.

c-5)PhotoLoader

`PhotoLoader` is a utility class used to manage `ImageIcon` objects.

- **Method: `getImage`:** This method returns a scaled `ImageIcon`. It uses the `Image.SCALE_SMOOTH` algorithm to ensure that characters and food icons look clear when resized to the 45x45 pixel grid cells. The icons are then assigned to the `JLabel` components inside the `GridCell` panels.

c-6) StartScreen

This class handles the initial configuration using modal Swing components.

- **Method: `showSetup`:** This method constructs a custom `JPanel` containing `JLabel`, `JComboBox`, and `JTextField` components. It uses a `GridLayout(0, 1)` to arrange these setup options in a single column within a `JOptionPane` dialog.
- **Method: `showGameInfo`:** This method uses a `JOptionPane.showMessageDialog` to present a static text manual to the user, providing details on roles and movement rules.

References:

- [1]<https://www.geeksforgeeks.org/java/java-swing-jfilechooser/>
- [2]<https://docs.oracle.com/javase/tutorial/uiswing/components/border.html>
- [3]<https://www.digitalocean.com/community/tutorials/serialization-in-java>
- [4] <https://gameprogrammingpatterns.com/type-object.html>
- [5] <https://www.baeldung.com/jcache>
- [6] https://www.tutorialspoint.com/swing/swing_layouts.htm
- [7] <https://www.geeksforgeeks.org/java/jlabel-java-swing/>
- [8] <https://docs.oracle.com/javase/tutorial/uiswing/events/mouselistener.html>
- [9] <https://www.geeksforgeeks.org/java/hashing-in-java/>
- [10] <https://docs.oracle.com/javase/tutorial/uiswing/components/dialog.html>
- [11] https://www.w3schools.com/java/java_date.asp